

コンピュータ アーキテクチャ I 第5回

塩谷 亮太 (shioya@ci.i.u-tokyo.ac.jp)

東京大学大学院情報理工学系研究科 創造情報学専攻

課題の解説

課題 4

- 10から1までを1つつ下りながら数える以下のループをアセンブリ言語で書け
 - 使用する命令セットは第2回の講義資料のものに準じる
- ヒント：
 - 減算には add の代わりに sub を使う
 - 初期値は li 命令で設定
 - 講義中の for 文の例のようにメモリ上に i を置くとややこしいので、レジスタの上で全て終わらした方が楽

```
1: for (i = 10; i > 0; i-=1) {  
2: }
```

■ C 言語

```
1: for (i = 10; i > 0; i-=1) {  
2: }
```

- そのままだと考えづらいので、
まず上記のループを下記の形に変換して考える

```
1:      i = 10;          // 初期化部分  
2: LABEL:                // ループの先頭  
3:      i = i - 1;       // カウンタの更新  
4:      if (i > 0)        // ループの継続判定  
5:          goto LABEL;  // LABEL に戻る
```

アセンブリ言語

```
// i = 10;
// レジスタ A に 10 をいれる
li 10→A
// if (i > 0) で使う 0 をセット
// B に 0 を読み込む
li 0→B
// 戻ってくるためにラベルを定義
LABEL:
// i = i - 1;
// A から 1 を引く
sub A,1→A
// if (i > 0)
//     goto LABEL;
b A>B, LABEL // 条件がなりたっていたら LABEL に飛ぶ
```

前回の振り返り

論理回路の作り方

- 以下のそれぞれの仕組みを使った回路を説明
 1. リレー
 2. CMOS
- これらの論理的な動作は実はほぼ同じに作れる
 - リレーの方が直感的にわかりやすいのでこちらから説明

NAND ゲートの動作

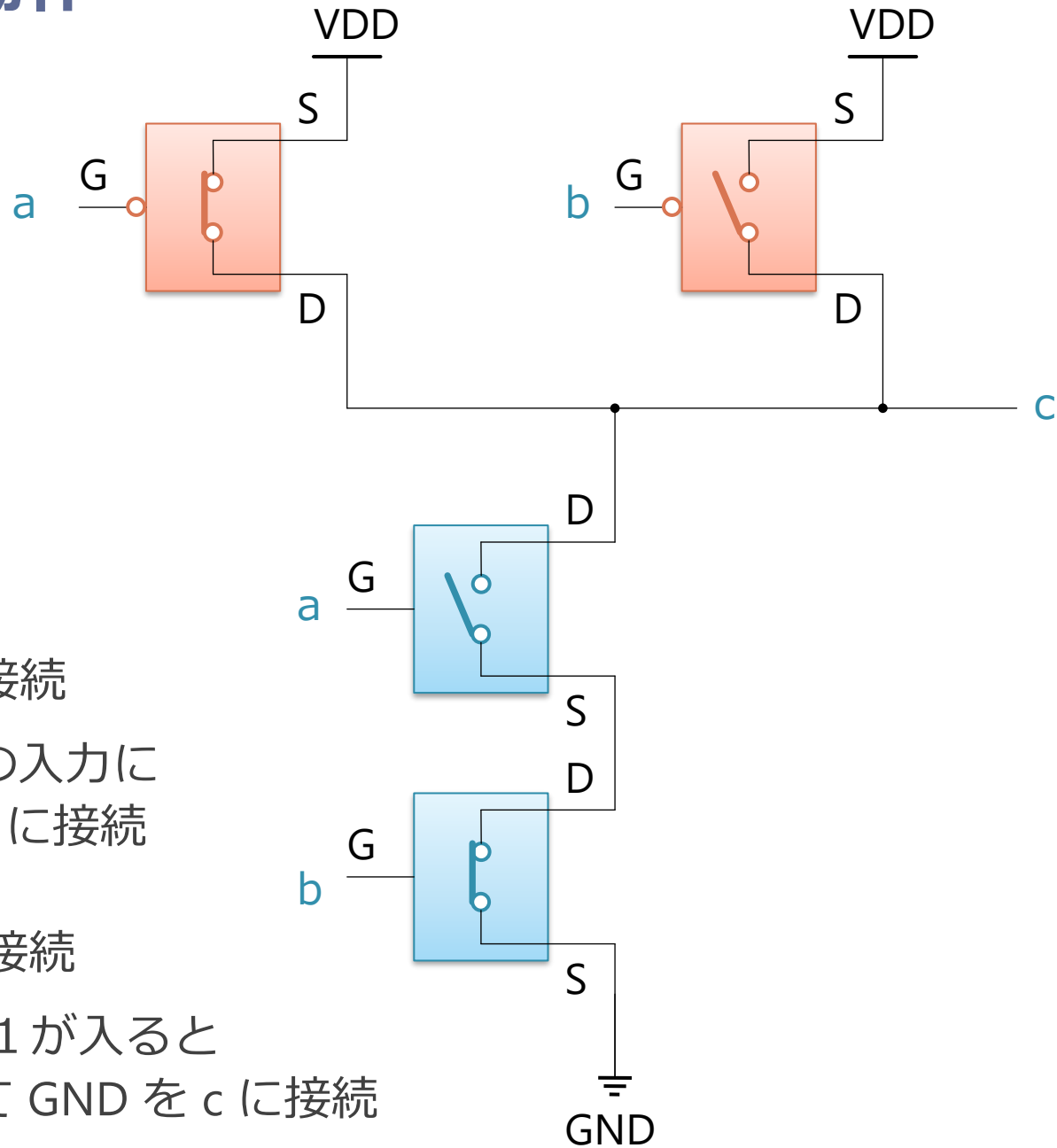
a	b	c
0	0	1
0	1	1
1	0	1
1	1	0

■ 上側の P 型 2 つは並列接続

- a と b どちらか片方の入りに 0 が入ると VDD を c に接続

■ 下側の N 型 2 つは直列接続

- a と b 双方の入りに 1 が入ると 2 つとも ON になって GND を c に接続



CMOS: Complementary Metal–Oxide–Semiconductor

- CMOS は NMOS と PMOS の 2 種類のトランジスタから成る
 - 電界で電荷（電子/正孔）を動かして ON/OFF する
- 基本的に N 型/P 型リレーとほぼ同じ動作
 - つまり全く同じように論理回路が組める
- 現代のコンピュータはこの CMOS を使って作られている

リレーがあればプログラムが実行できる

- コンピュータがあれば、プログラムが実行できる
 1. コンピュータは機械語を解釈して実行できる
 2. 機械語はアセンブリ言語から変換できる
 3. アセンブリ言語はC言語からコンパイルできる
- 途中に多数のステップがあるが、まとめると,
 - リレーがあれば、C言語のプログラムを実行できる

一般化すると,

- なんらかの入力に従って ON/OFF できるスイッチがあれば, それでプログラムが実行できるコンピュータが作れる
 - 具体的な回路の組み方は, 回路の種別ごとに違う事もある
 - NAND さえ出来ればこっちのもの

命令パイプライン

導入：工場のラインを考える



- ベルトコンベアのラインの上を製品が流れていく
 - 4 人の人が、それぞれの工程の作業をおこなって完成
- 上のように1つしか製品をながさないで、
 - 各人は他の人が作業している間はヒマ

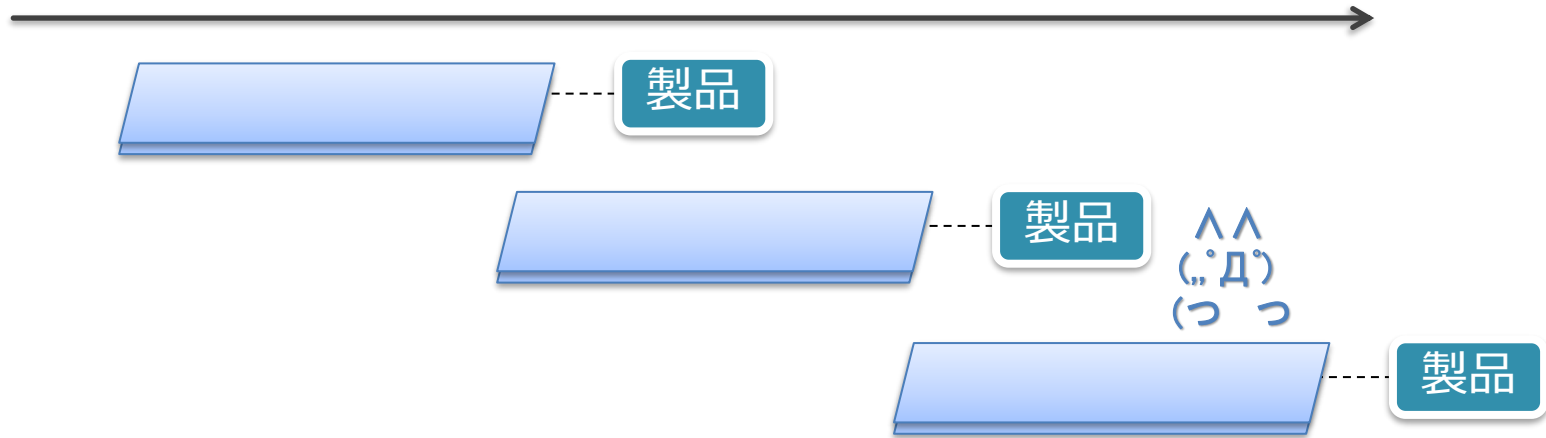
導入：工場のラインを考える



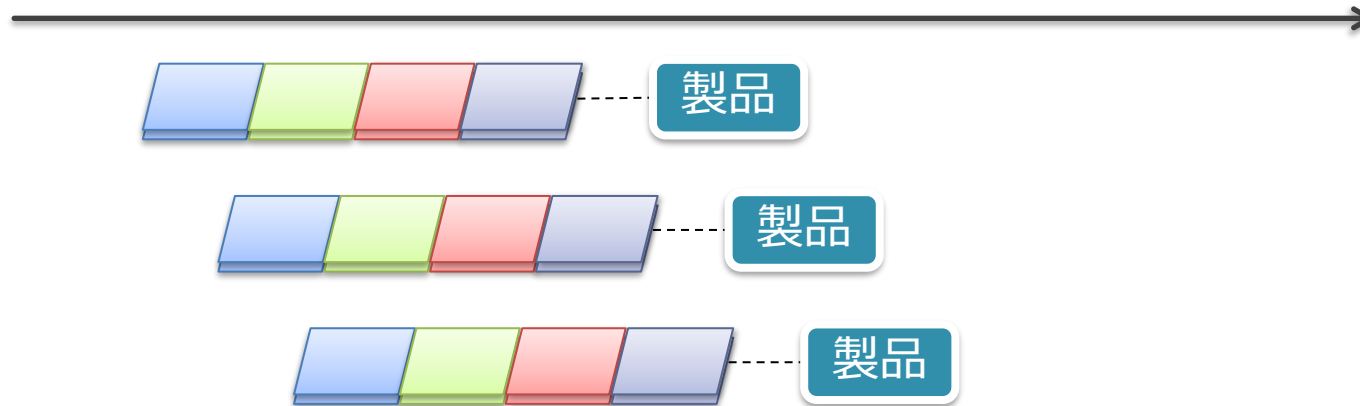
- 実際の工場：複数の製品を同時に流す
 - 各工程を並列して処理することによりスループットを向上
 - さっきの4倍の速度で製品ができあがっていく
- これが 命令パイプライン

パイプライン化による性能向上

パイプライン化しない場合



パイプライン化した場合



今日の内容：命令パイプライン

1. シングル・サイクル・プロセッサの動作
 - 全ての命令の処理が1サイクルで完結
 - これまでに説明していたプロセッサの動作と同じ
 - パイプライン化を前提とした, より詳細なものを使って復習
2. 上記のパイプライン化
 - 具体的にどうパイプライン化するか
3. パイプライン化の性能への影響

シングル・サイクル・プロセッサの動作

もくじ

1. シングル・サイクル・プロセッサの動作
2. 上記のパイプライン化
3. パイプライン化の性能への影響

これまでに説明した CPU のイメージ

■ コンピュータの心臓部

- メモリから命令を読み出し，計算する

■ 構成要素：

● 演算器（FU: Functional Unit）

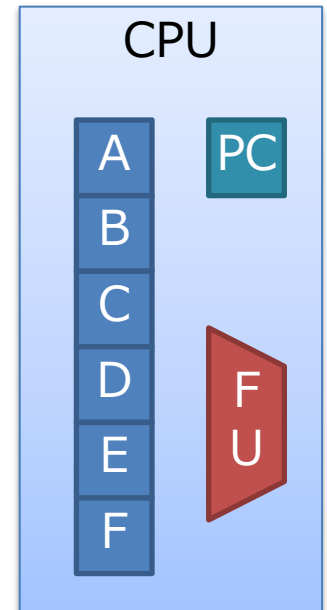
- 加算器や AND 演算器など
- 指示された種類の演算を行う

● レジスタ・ファイル（右図では A,B,C...）

- メモリと同様にデータを記憶する
 - ◇ 位置を指定して読み書きする
- CPU の演算は，このレジスタ上でのみ行う

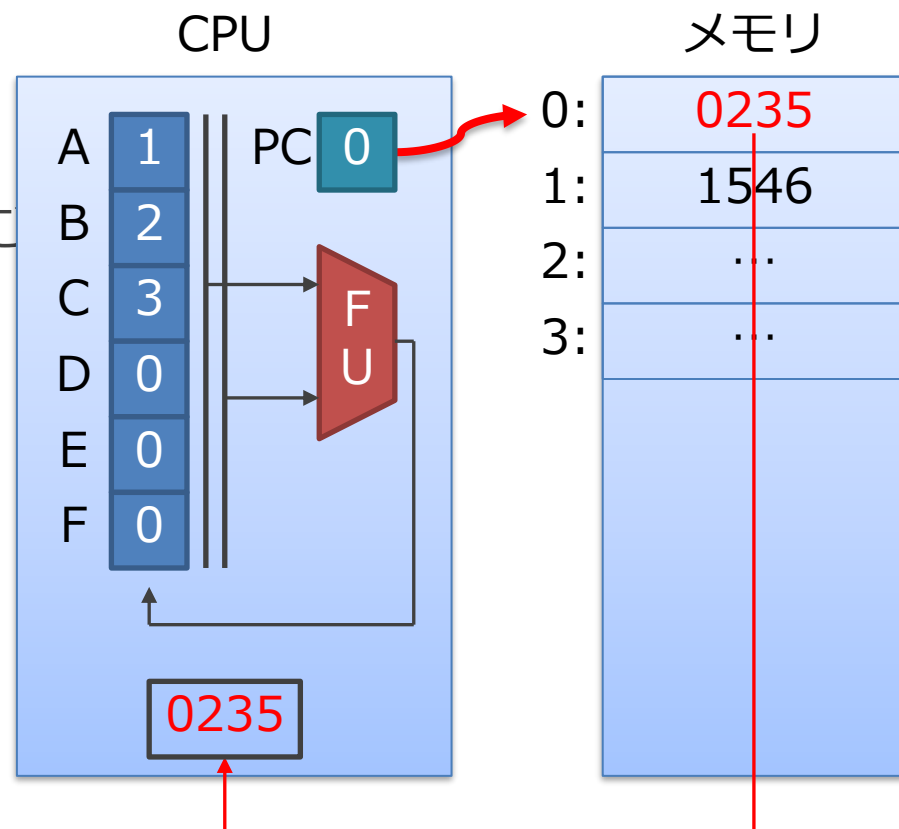
● PC（Program Counter）

- 現在見ている命令のアドレスを記憶している場所

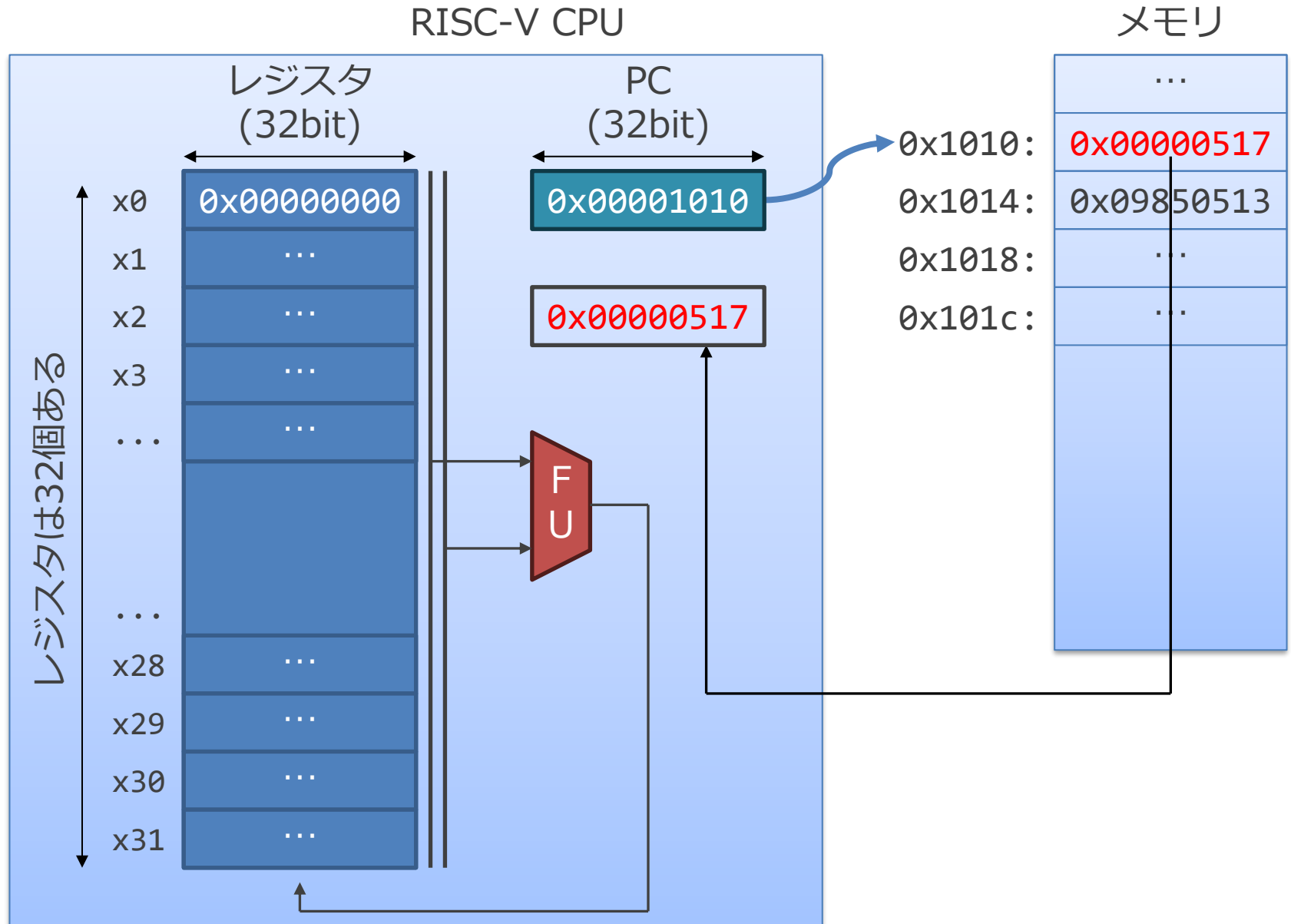


これまでに説明した CPU のイメージ

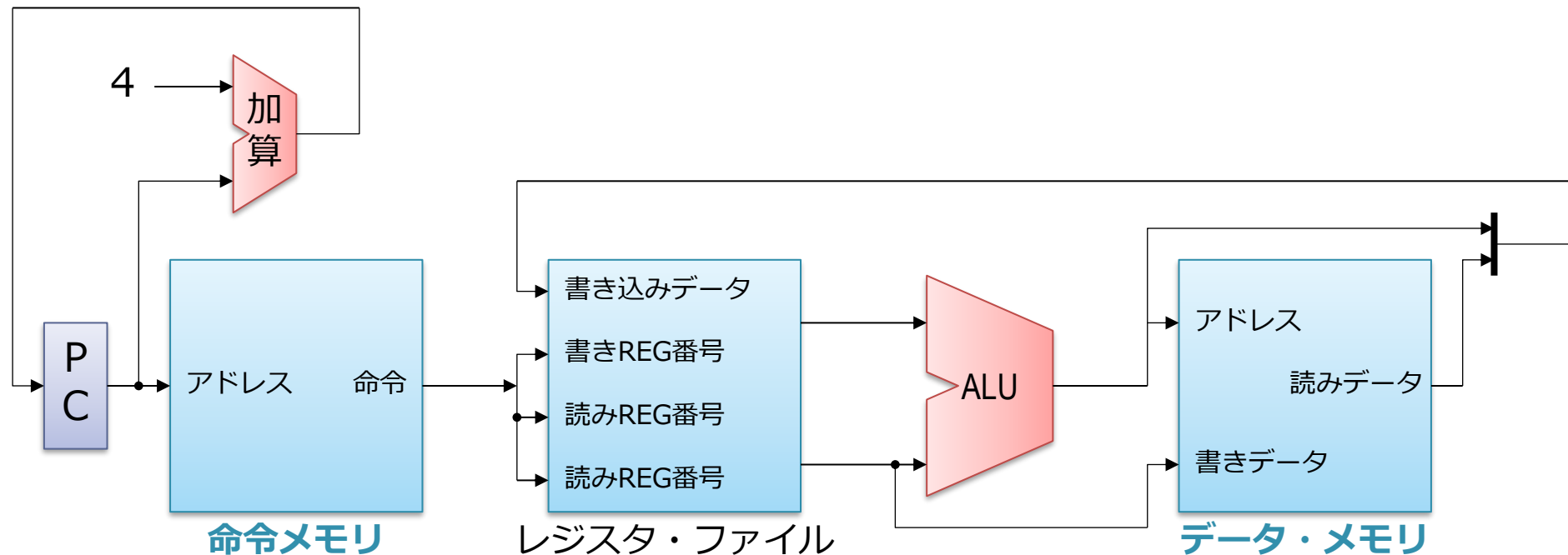
1. PC が指している命令の番地を読む
 1. 今はアドレス 0 を指している
2. 内容である 0235 が得られる
3. CPU 内にもってくる



これまでに説明した RISC-V (32bit) のイメージ



ベースとなるシングル・サイクル・プロセッサ



■ 以前説明したものとの違い：

- メモリが**命令メモリ**と**データメモリ**に別れている
- 算術 & 論理演算, ロード, ストアのみを実行可能
 - 分岐とジャンプは, 簡単のために今は考えない

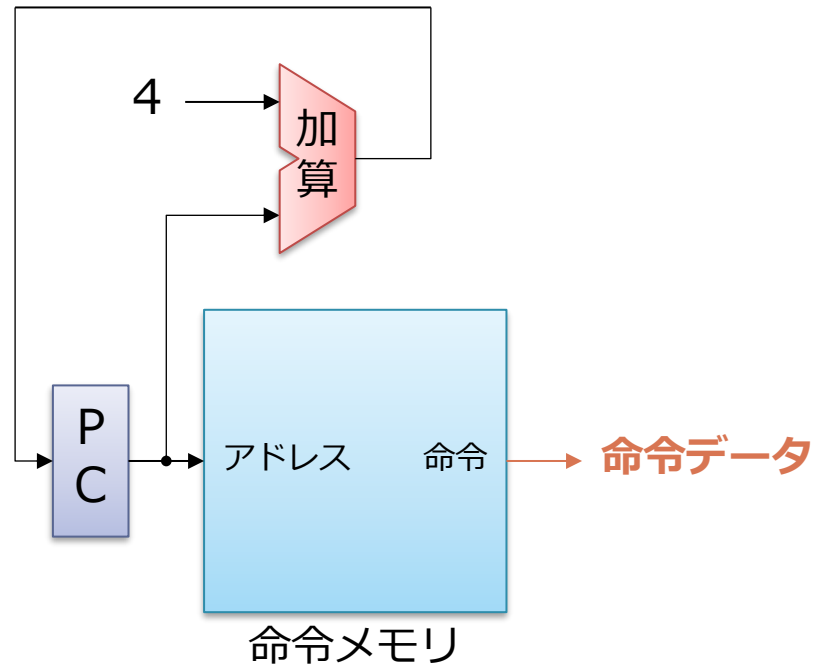
1命令の実行フェーズ

■ 実行フェーズ

- フェッチ
- デコード
- レジスタ読み出し
- 実行
- レジスタ書き戻し

■ RISC-V の加算命令を実行する流れをざっとみる

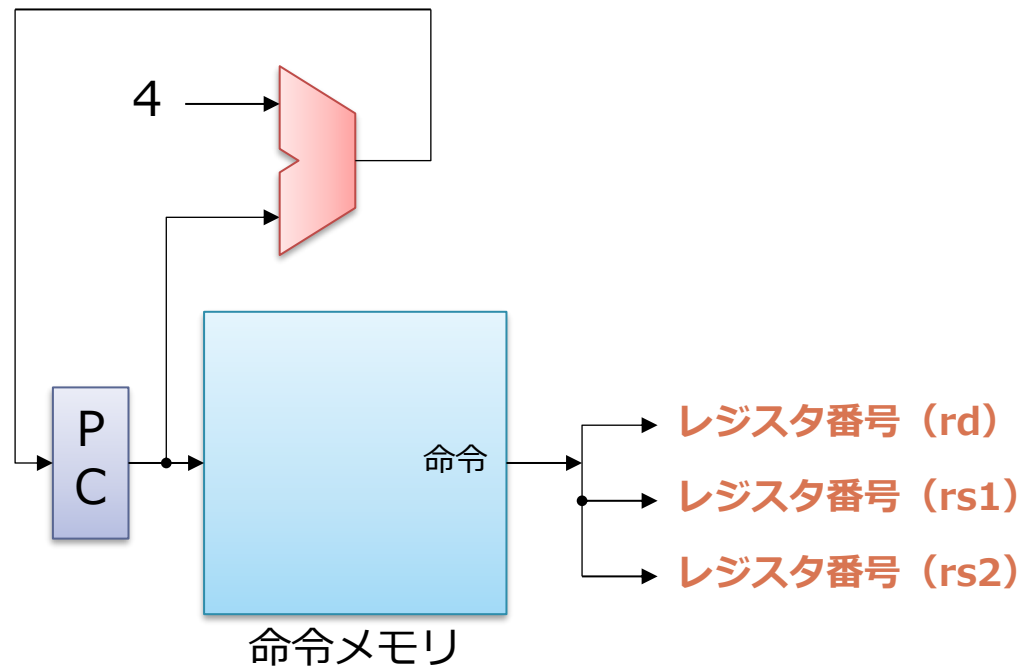
命令フェッチ



■ 命令メモリから命令を読み出す

- 命令メモリを順に読んでいくため、PC は毎サイクル加算される
- 足している 4 は、RSIC-V では命令の幅が 4 バイトだから
- 基本的に、この部分はどの命令でも変わらない

命令デコード

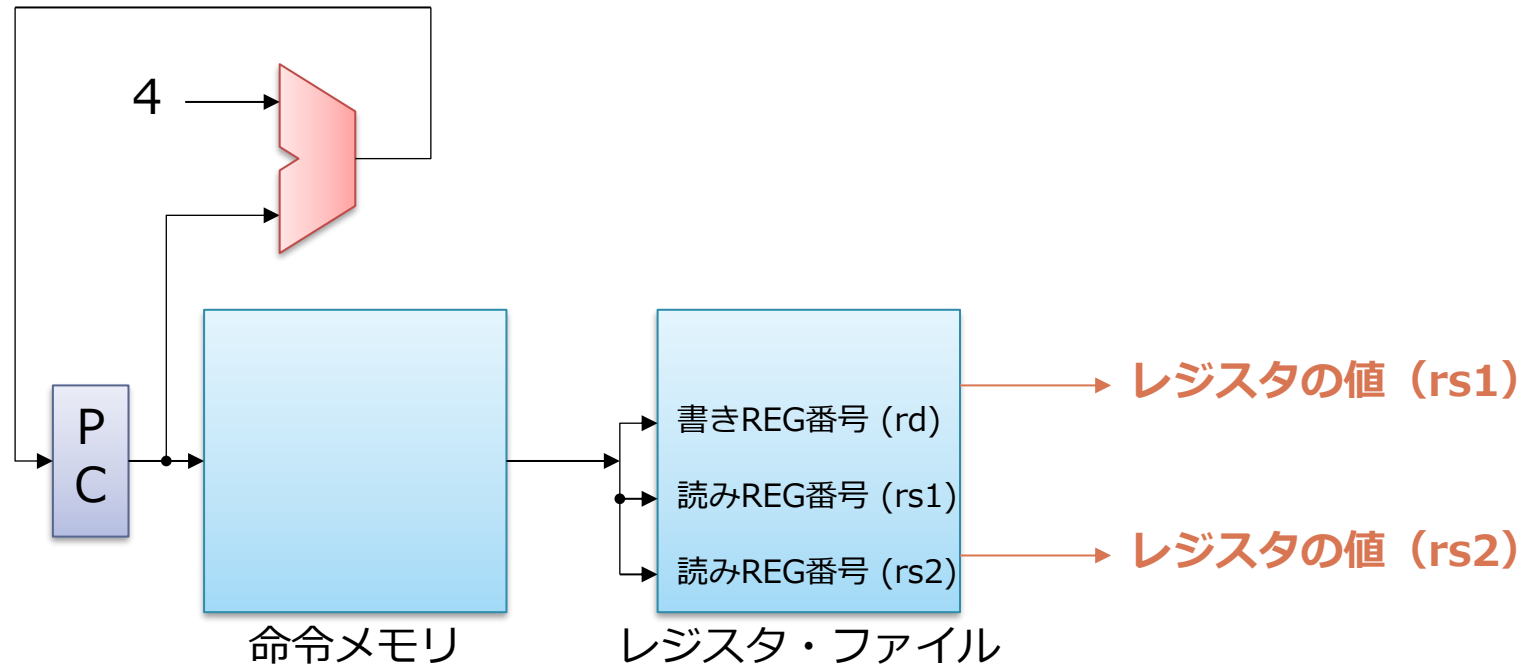


ADD : $x[rd] \leftarrow x[rs1] + x[rs2]$



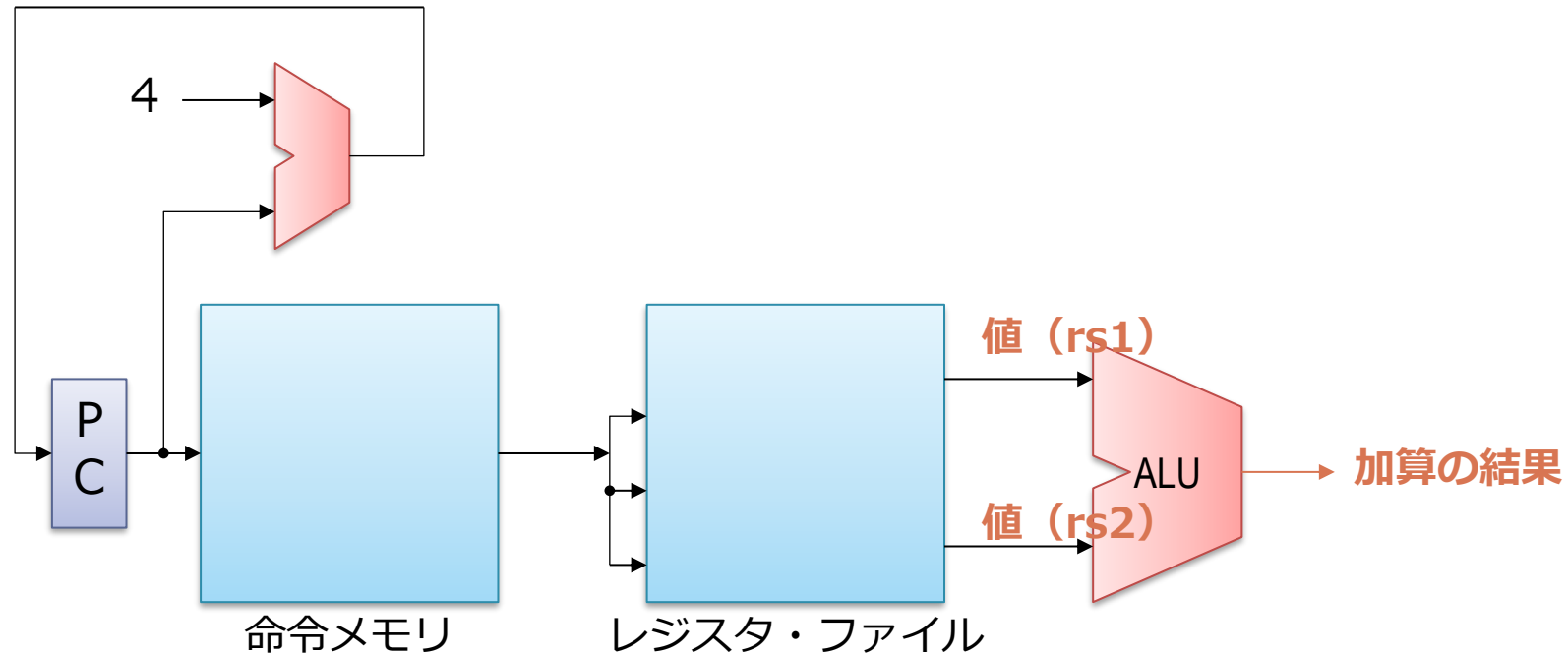
- 取り出した命令からレジスタ番号を表す部分のビットを取り出す
 - ソース (rs1, rs2) とディスティネーション (rd)

レジスタ読み出し



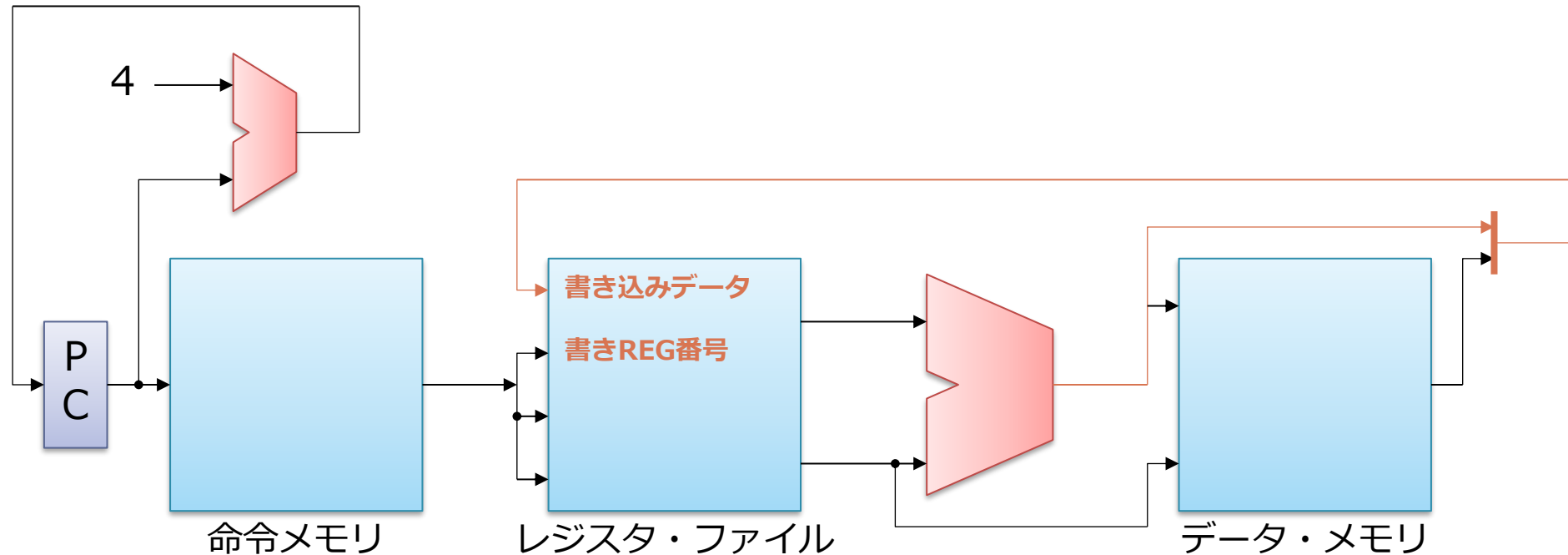
- デコードで得られたレジスタ番号を使って RF にアクセス
 - ソース・オペランドの値を読み出す

実行



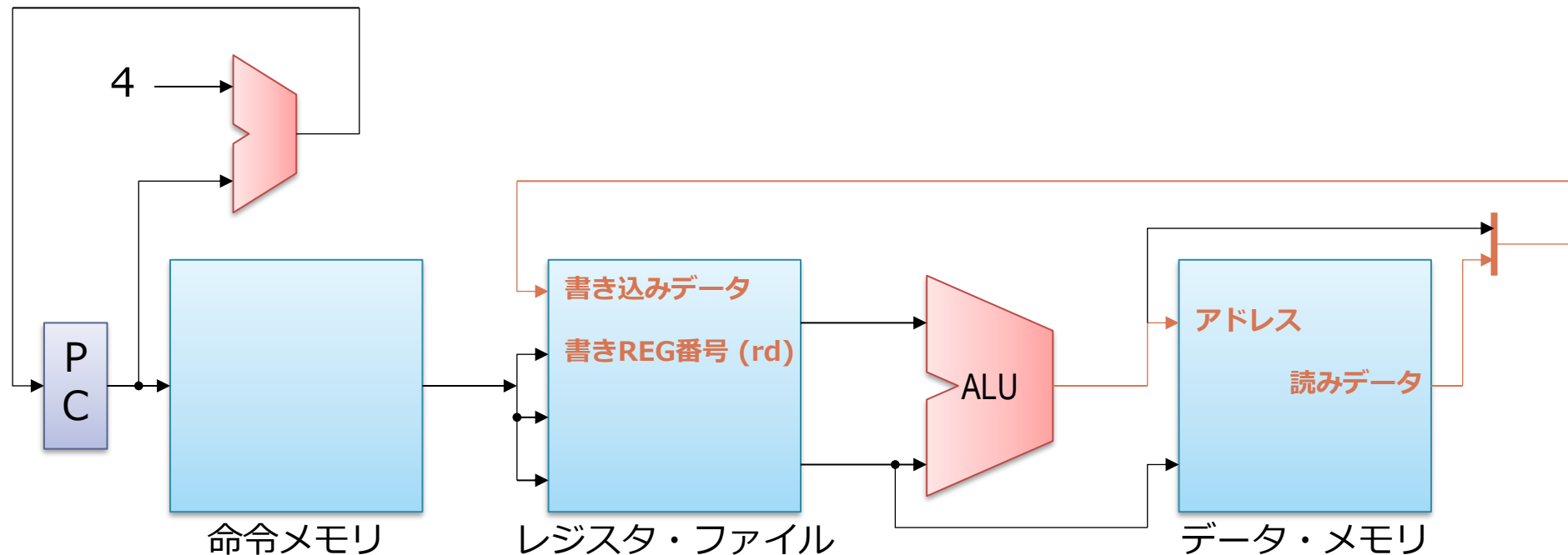
- RF から読みだした 2 つの値を加算

レジスタ書き戻し



- 加算の結果をレジスタ・ファイルに書き戻す
 - データ・メモリには用がないので何もしない

ロードの場合：メモリ・アクセスが加わる



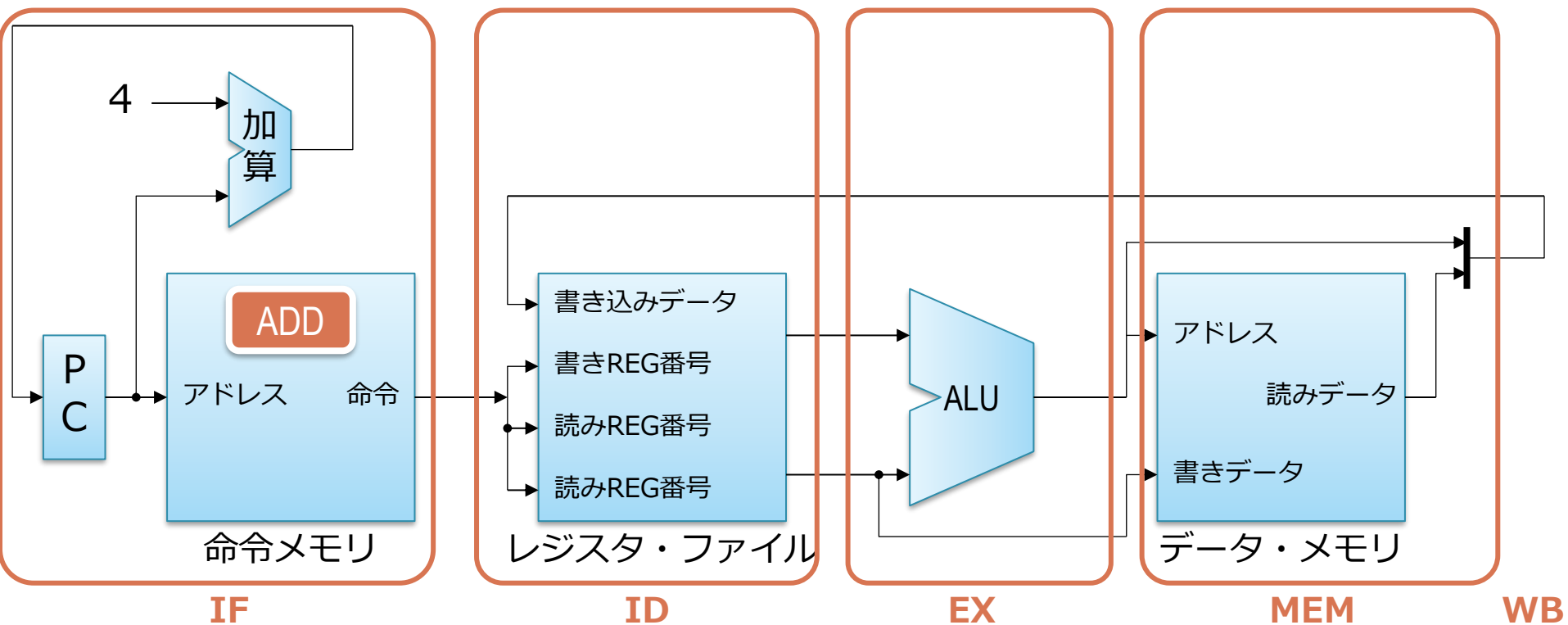
LW : $x[rd] \leftarrow (x[rs1] + \text{immediate})$



■ 加算命令との違い：

- アドレスの計算 ($x[rs1] + \text{immediate}$) を ALU でやる
- 得られたアドレスでデータ・メモリにアクセス

各処理は基本的には左から右に流れる



- 特定のユニットで仕事をしている間，他の部分は遊んでいる
- パイプライン化
 - これをもとに，導入で話したように処理をオーバーラップさせる

パイプライン化

もくじ

1. シングル・サイクル・プロセッサの動作
- 2. 上記のパイプライン化**
3. パイプライン化の性能への影響

パイプライン化

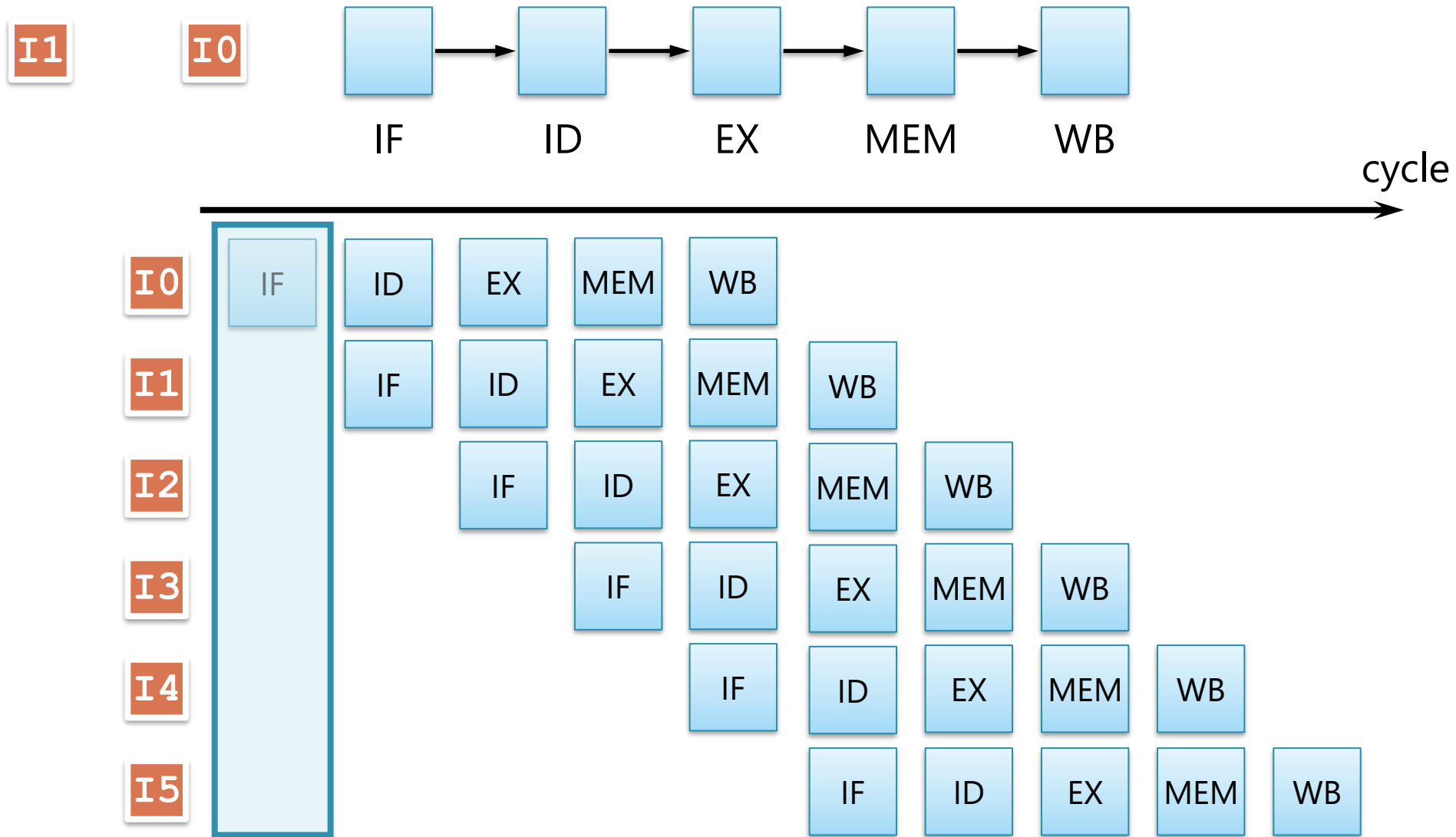
- 回路のまとまりをオーバラップさせる単位にする
 - この単位をステージと呼ぶ
- ステージ
 1. IF : 命令フェッチ
 2. ID : デコードとレジスタ読み出し
 3. EX : 実行
 4. MEM : メモリ・アクセス
 5. WB : レジスタ書き込み

工場のラインを考える（再）



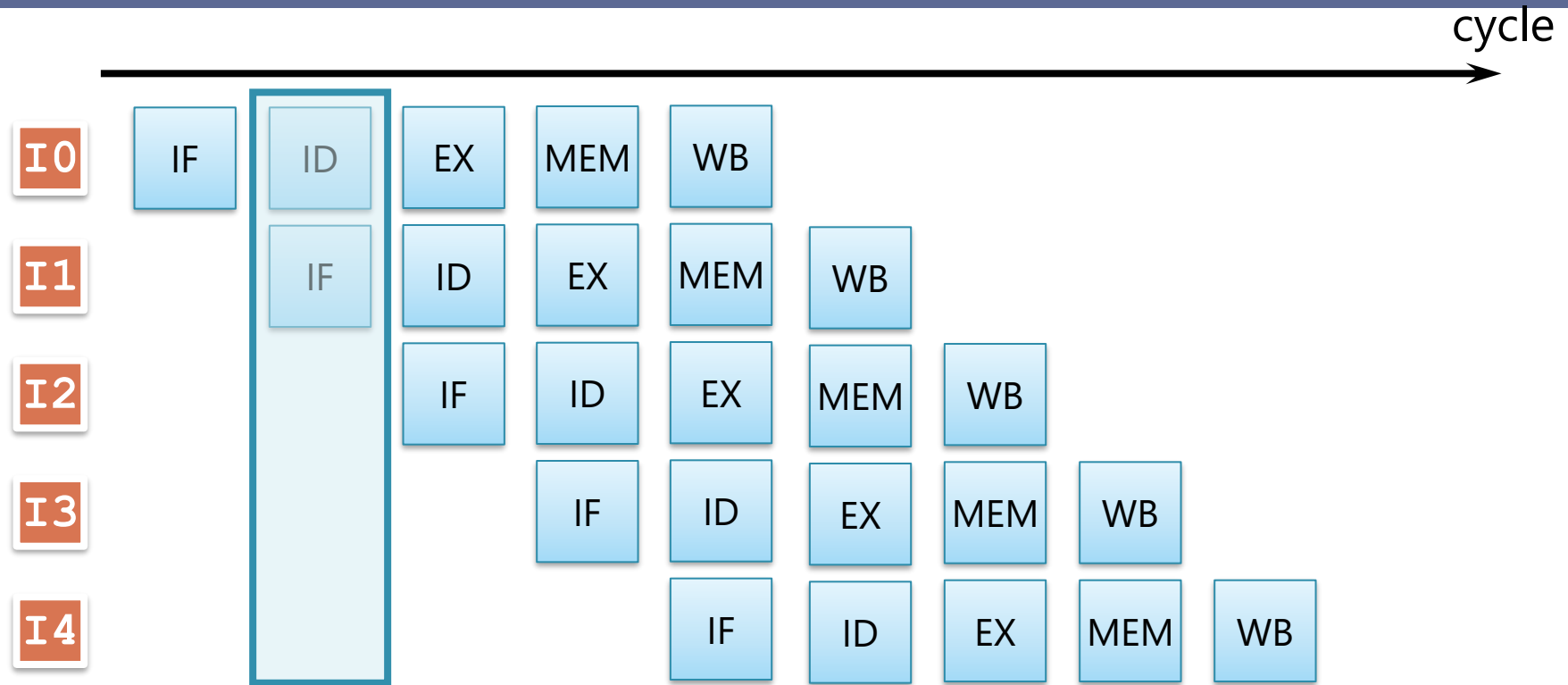
- 実際の工場：複数の製品を同時に流す
 - 各工程を並列して処理することによりスループットを向上
- これが 命令パイプライン

命令パイプラインの実行の様子



パイプライン・チャートの見方

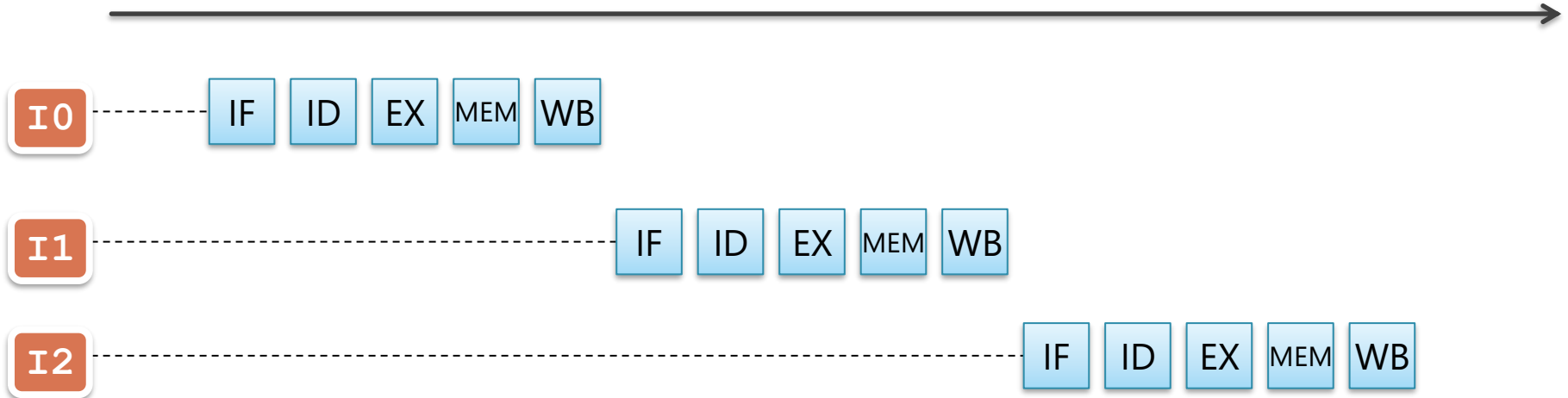
ここから先で多用されるので重要



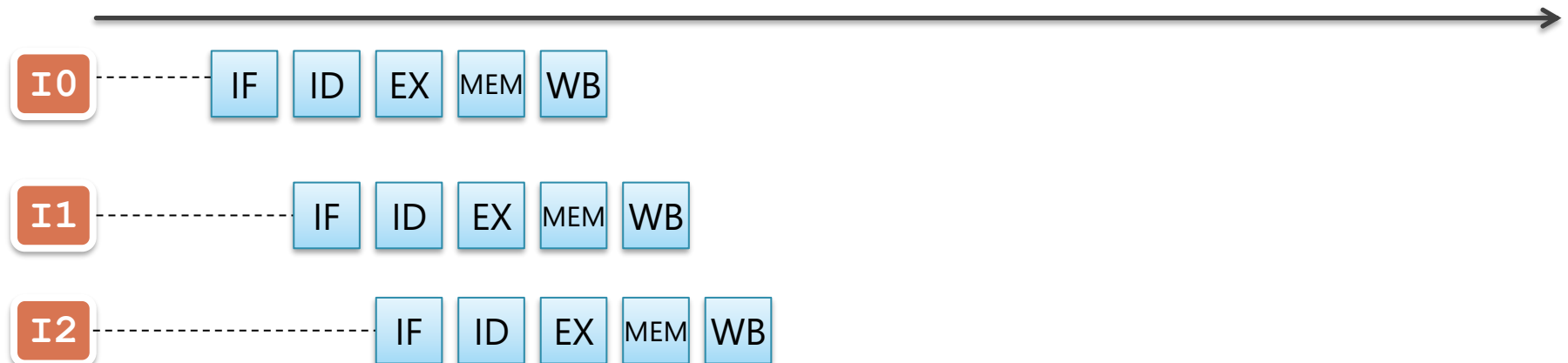
- 左から右にむかって時間は進む
- 上から下にむかって命令が実行順に置かれる
- 各ステージを表す四角は左側にある命令がその時そこにいることを示す
 - 上記では2サイクル目に、I0 が ID に、I1 が IF で処理されている

パイプライン化による性能（スループット）向上

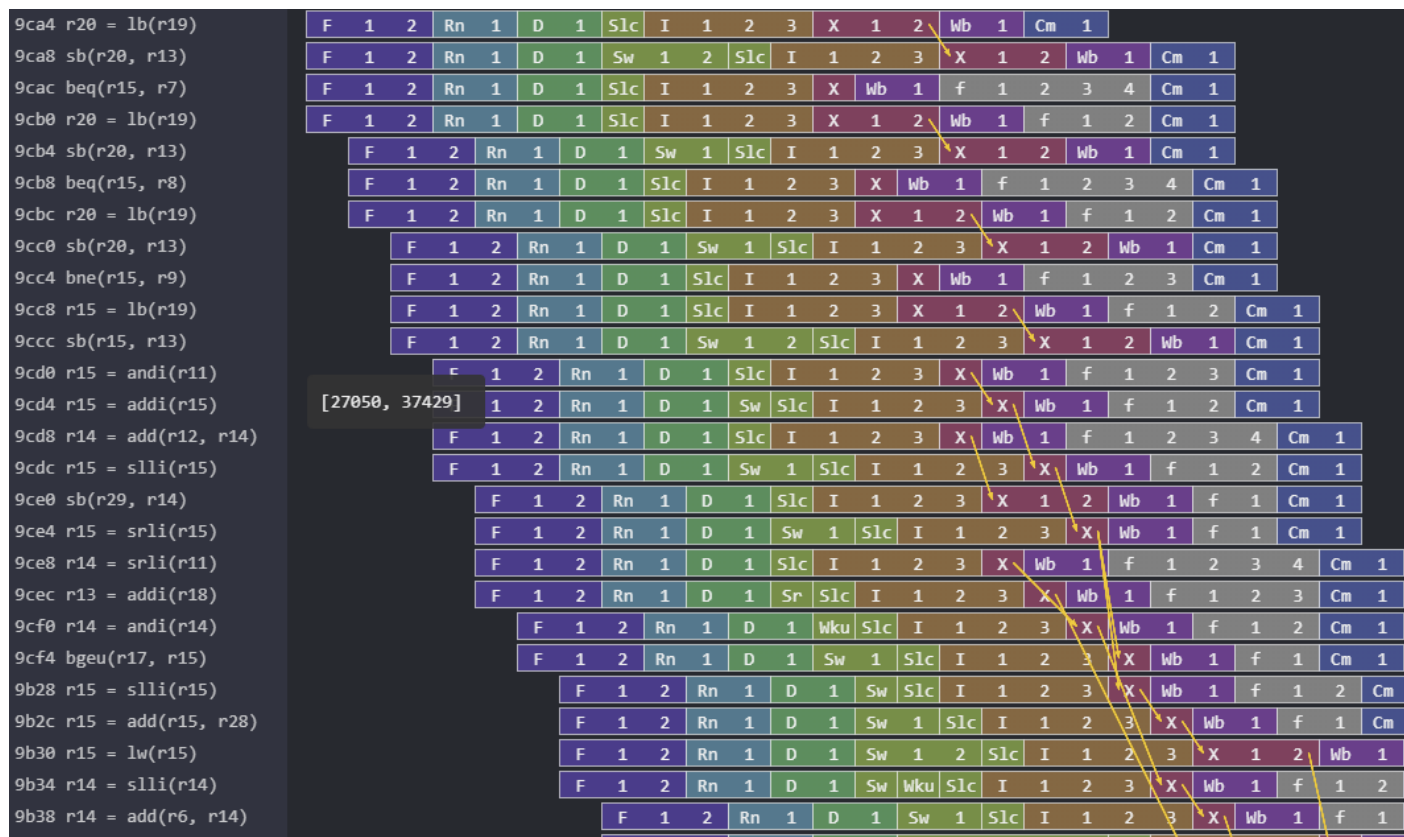
パイプライン化しない場合



パイプライン化した場合



余談：実際の CPU を実行した場合のパイプライン

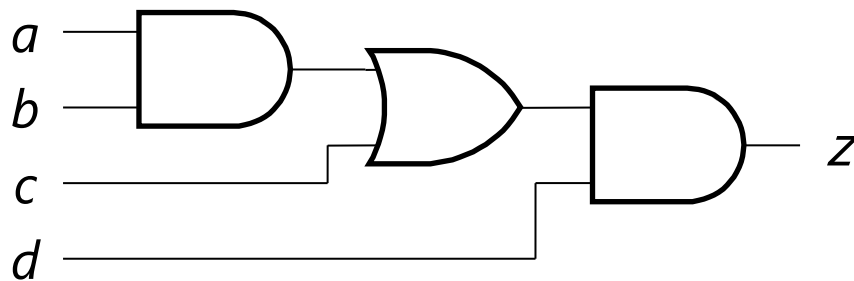


■ 塩谷が開発している RISC-V CPU（RSD）の実行を可視化したもの

- <https://github.com/rsd-devel/rsd>
- out-of-order 実行をしているので、途中からプログラム順とは異なるタイミングで実行が進んでいる

ステージを「どうやって」切るか

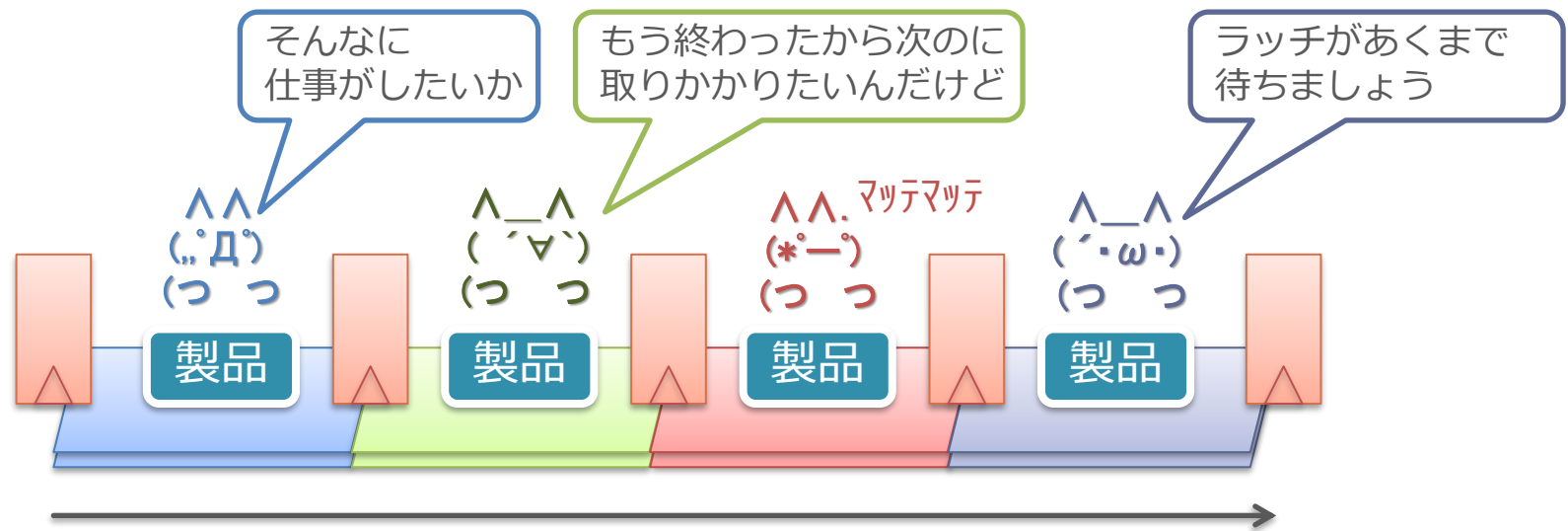
- シングル・サイクル・プロセッサの回路に適当に間隔をあけて命令を流せばよいというものもない
- 1. 各ステージを完全に同じ長さにするのは凄く難しい
 - 同じ長さ=同じ遅延=全く同じ段数の組み合わせ回路
- 2. 長いステージであっても信号は絶えず変化する可能性がある
 - 短いパスから順に出力に反映される
 - たとえば下の回路で a, b, c, d が全て変化したとすると、まず d の変化が z に反映し、次に d が...



パイプライン化（オーバーラップ）の実現方法

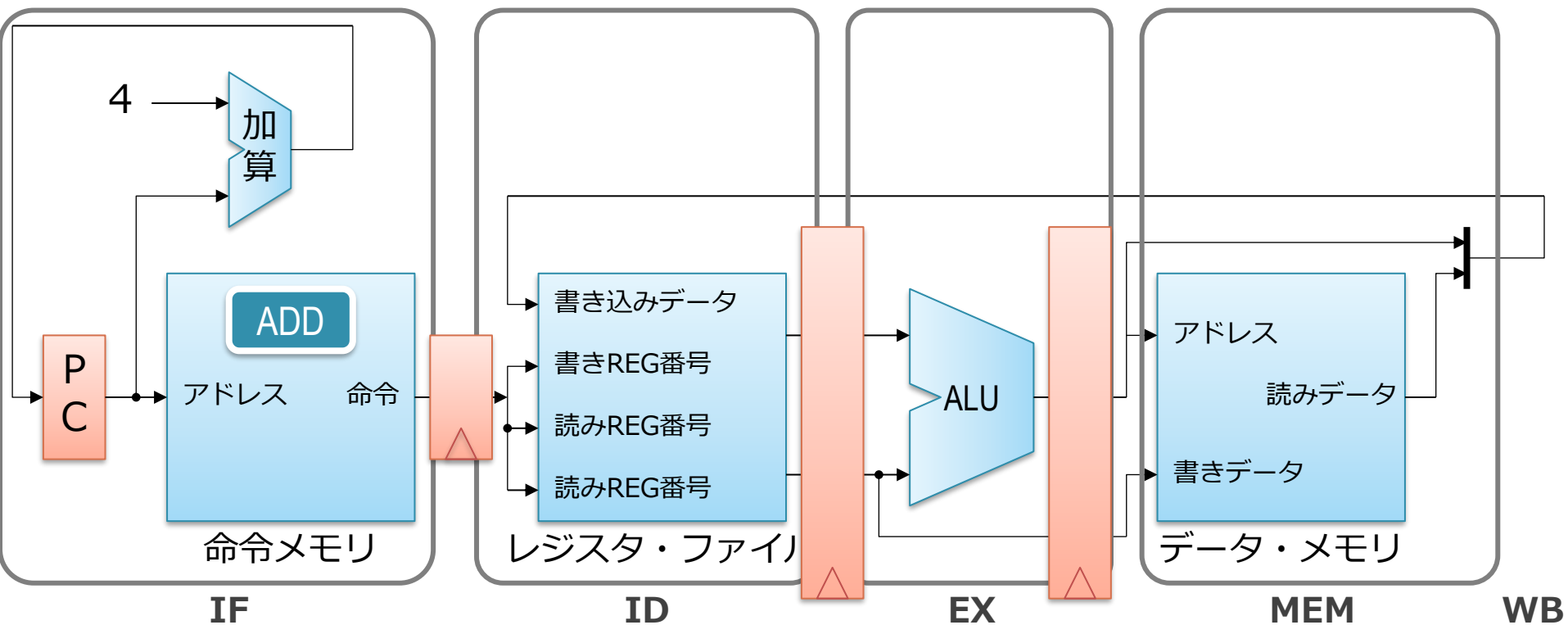
- シングルサイクル・プロセッサにフリップ・フロップをいれる
 - このためのフリップ・フロップをパイプライン・ラッチという
 - ラッチ (latch) は「ドアや門などの掛け金」の意味
 - パイプライン・ラッチで区切られた部分がステージとなる
- 1 サイクルの間はパイプライン・ラッチで信号がとまる
 - 複数ステージ間で信号が混じるのを防ぐ
 - 指定された時間までラッチでドアを開かなくするイメージ？

パイプライン・ラッチのイメージ



- 各人の作業が終わっても，ラッチが開くまでは次の人に製品を送れない
 - 複数ステージ間で信号が混じるのを防ぐ
 - 指定された時間までラッチでドアを開かなくするイメージ？
 - 上の絵では，赤い人がまだ仕事が終わってないので，全体が歩調を合わせるために待っている

パイプライン化（オーバーラップ）の実現方法



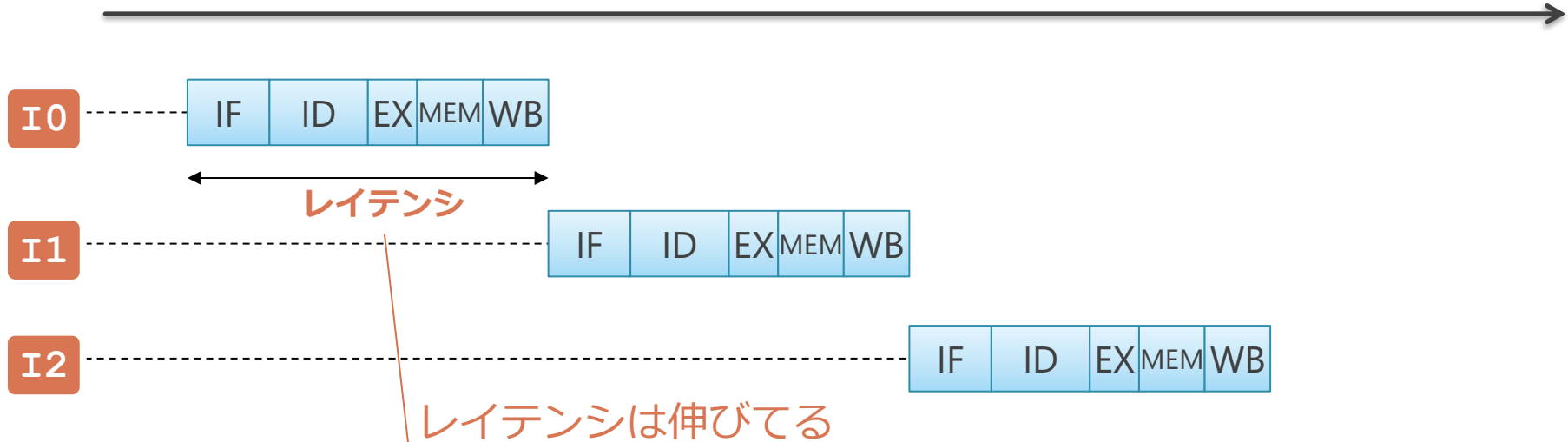
- 各ステージの間に, D-FF (オレンジの四角) を入れる
 - WB の書き込みについては, レジスタ・ファイル自体がクロックに同期して書き込みが行われるので D-FF は不要
- 各ステージの処理が早く終わっても, 次のクロックまでは D-FF で信号の伝搬を止める

パイプライン化の効果

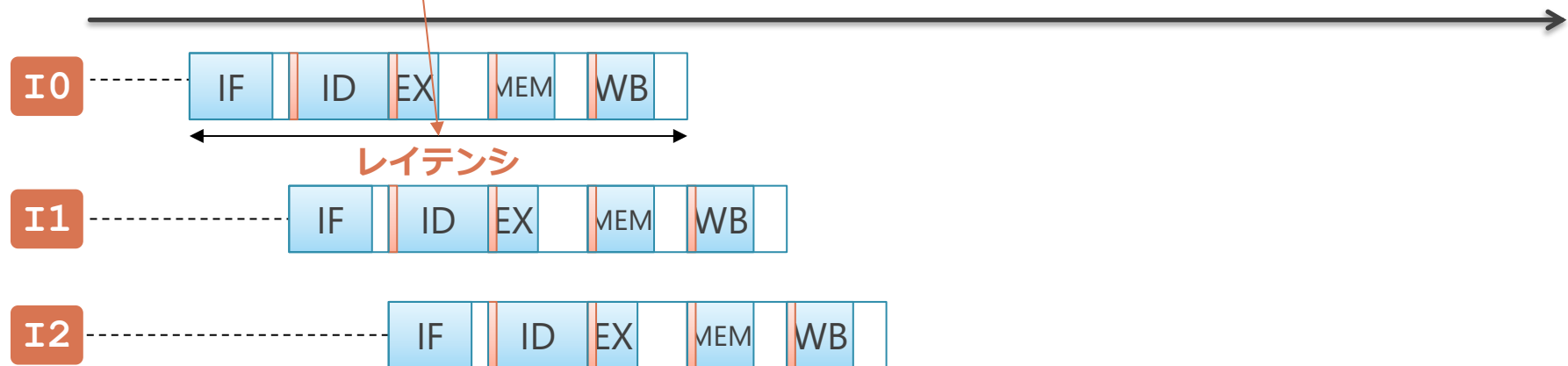
- レイテンシ (latency) : 短くならない (か, やや延びる)
 - 一続きの処理が始まってから終わるまでにかかる時間
 - この場合, 1命令の始まりから終わりまでの処理時間
 - 一番長い処理に合わせて動かすので伸びる
 - 原理的に短くならない
(ステージ間にフリップフロップが入る分は絶対のびる)
- スループット (throughput) : ステージ数倍だけ上がる
 - 単位時間当たりの処理量
 - この場合, 単位時間あたりに実行される命令数

パイプライン化の効果 レイテンシは伸びるが、単位時間あたりの処理命令数は増える

パイプライン化しない場合

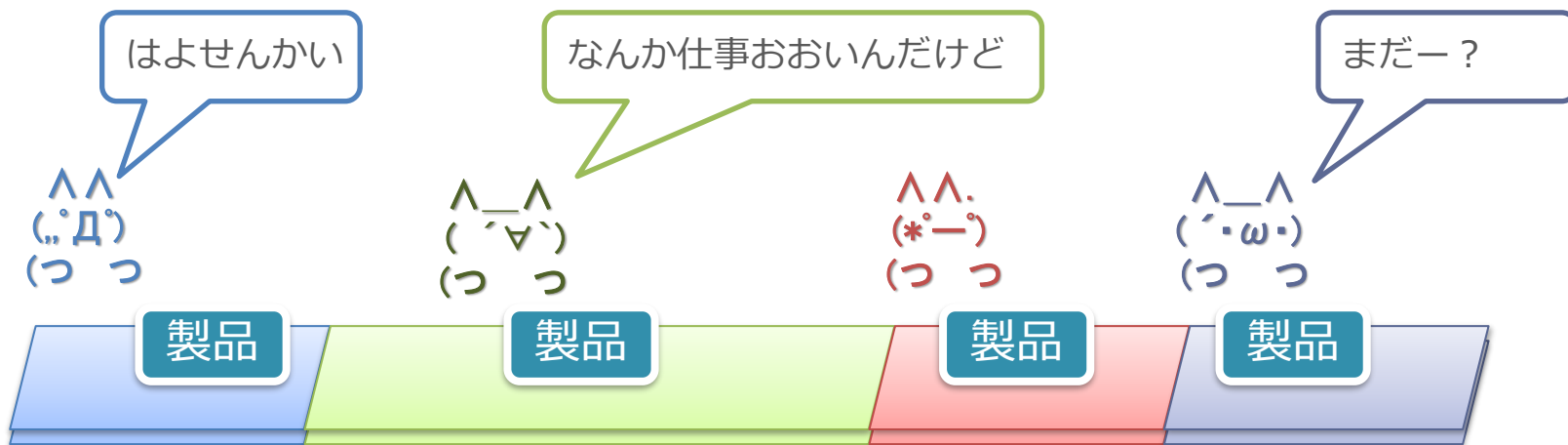


パイプライン化した場合



ステージを「どこで」切るか

- 大きな回路のまとまりをステージにする
 - 回路のまとまりが大きい → 遅延も大きい
- この遅延の大きさが揃っていないと、綺麗にうごかない
 - パイプライン全体は、一番遅いステージの遅延にあわせて動く
 - 他の人が仕事が終わったからと言って、先に送れない
- 良くない例：緑の人だけ仕事が多いので、全体が動かせない



ステージを「どこで」切るか

■ ステージ

1. IF : 命令フェッチ
2. ID : デコードとレジスタ読み出し
3. EX : 実行
4. MEM : メモリ・アクセス
5. WB : レジスタ書き込み

■ 上記では、デコードとレジスタ読み出しが ID ステージにまとめられている

- デコードにかかる遅延はほとんどない
- 読み出した命令からオペランドを取り出すのは、単に信号線を繋ぐだけで良い

余談：非同期回路やウェーブ・パイプライン

- クロックによる同期化を使わずにパイプラインを作る方法もあるにはある
- やり方：
 1. 色々な方法でステージ間の遅延の大きさを気合いで揃える
 2. 一定間隔でデータを流す
- 設計 & 動作させることがすごく難しいので、主流ではない
 - 特に、高速動作がかなり難しい

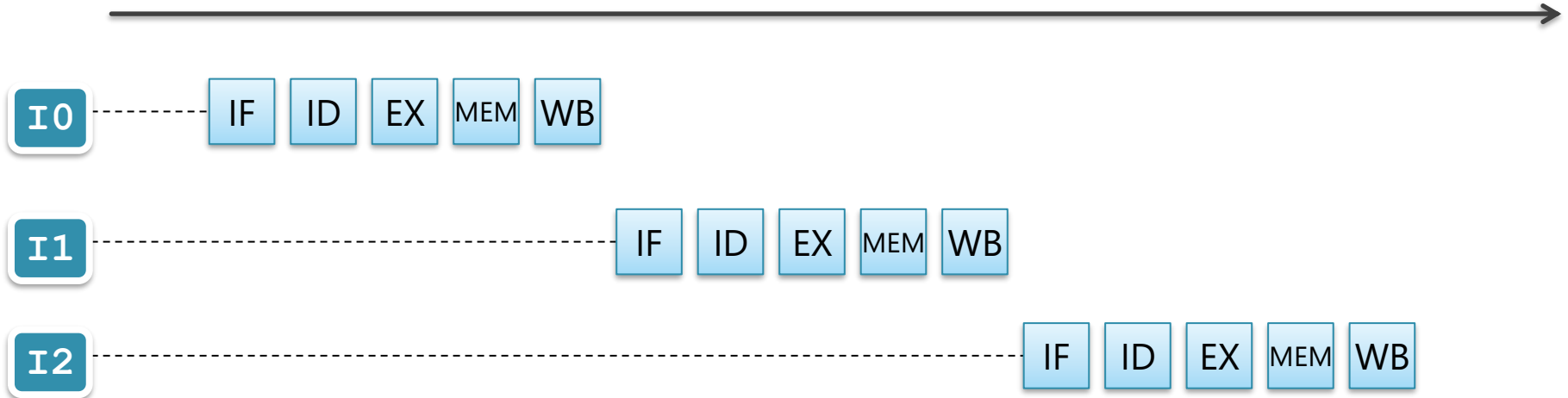
パイプライン化の性能への影響

もくじ

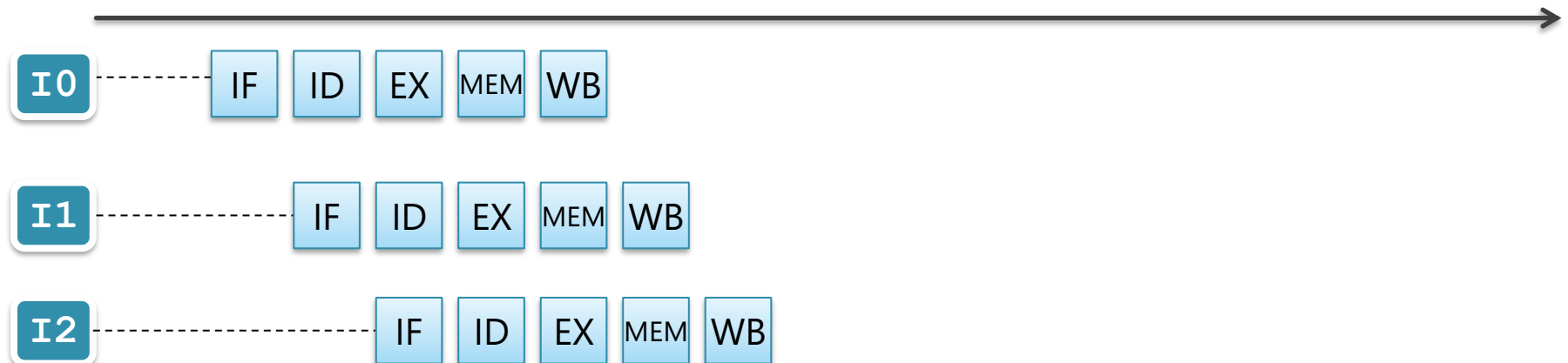
1. シングル・サイクル・プロセッサの動作
2. 上記のパイプライン化
3. **パイプライン化の性能への影響**

パイプライン化によるスループット向上

パイプライン化しない場合



パイプライン化した場合



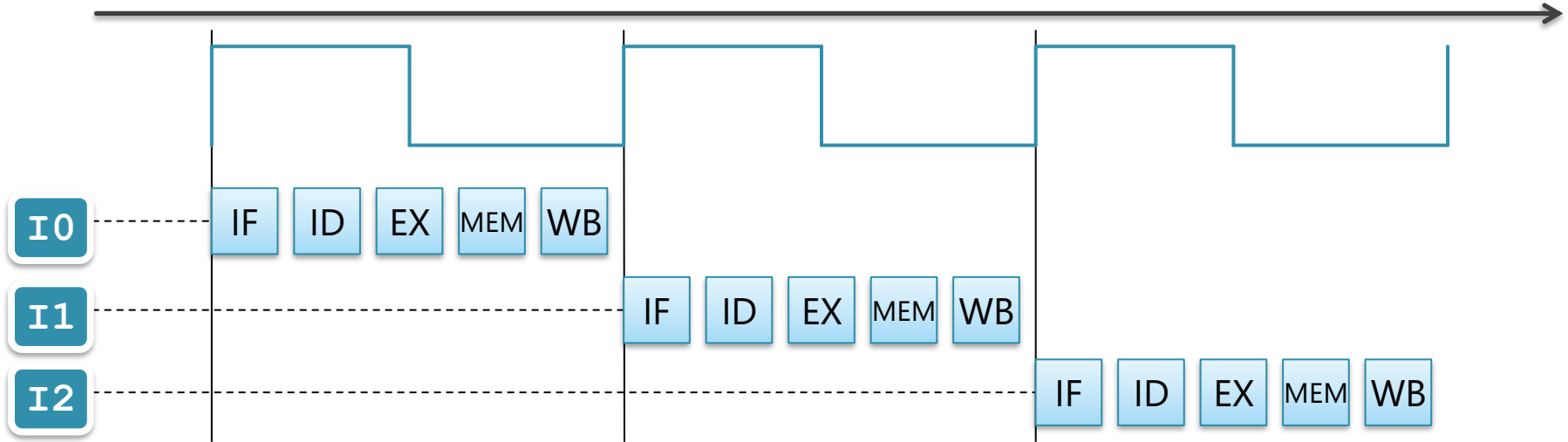
パイプライン化の意味

- パイプライン化の効果：
 - スループットの向上
 - = 単位時間あたりに処理できる命令の数の増加
 - = 動作クロック周波数の向上
- これらは同じ事を言い換えてるだけ

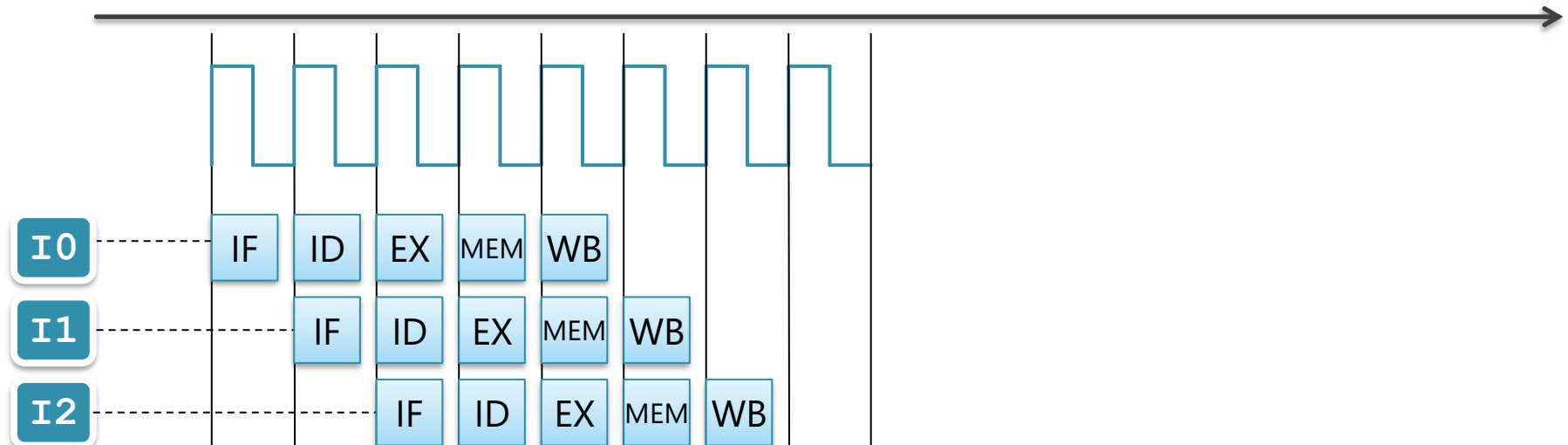
パイプライン化によるクロック周期の短縮

クロックの立ち上がりごとに、1命令が処理

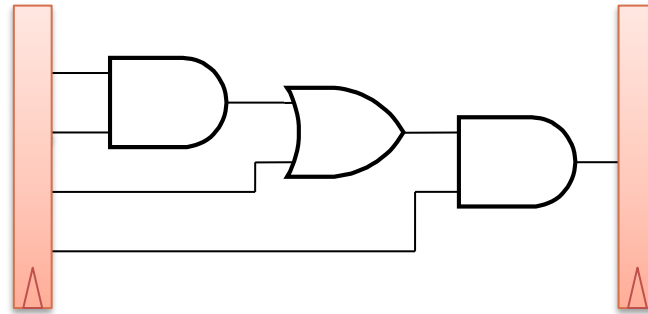
パイプライン化しない場合



パイプライン化した場合



ステージ内の信号の伝播を考える



■ パイプライン：ステージ：

- 複数のパイプライン・ラッチで挟まれてた,
- 組み合わせ回路（ゲート）

■ 矢印の動き：

- クロック開始時に、左のラッチからでた信号が
- 組み合わせ回路を通して、伝播していく様子

2 段にパイプライン化した場合

パイプライン化せず



2 段パイプライン



■ クロック周波数は 2 倍に :

- 各矢印の伸びる速度（信号が伝播する速度）自体は同じ
- 2 段パイプラインでは, ラッチから 2 回信号が出ている

4段にパイプライン化した場合

パイプライン化せず



2 段パイプライン



4 段パイプライン



■ クロックが4 倍に

- 4 段パイプラインでは, ラッチから 4 回信号が出ている

パイプライン化の限界



- パイプライン段数を増やしていけば、どこまでも速くなるのか？
 - ならない
- 理由：
 1. 回路的な理由による周波数向上の限界
 2. アーキテクチャ的な理由による実効性能の限界

回路的な理由

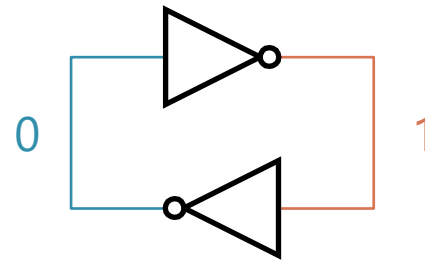
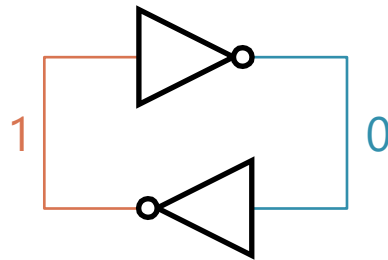
■ 理由：

1. D-FF 自体の遅延のため（発展）
2. 消費電力と熱のため

記憶素子の原理

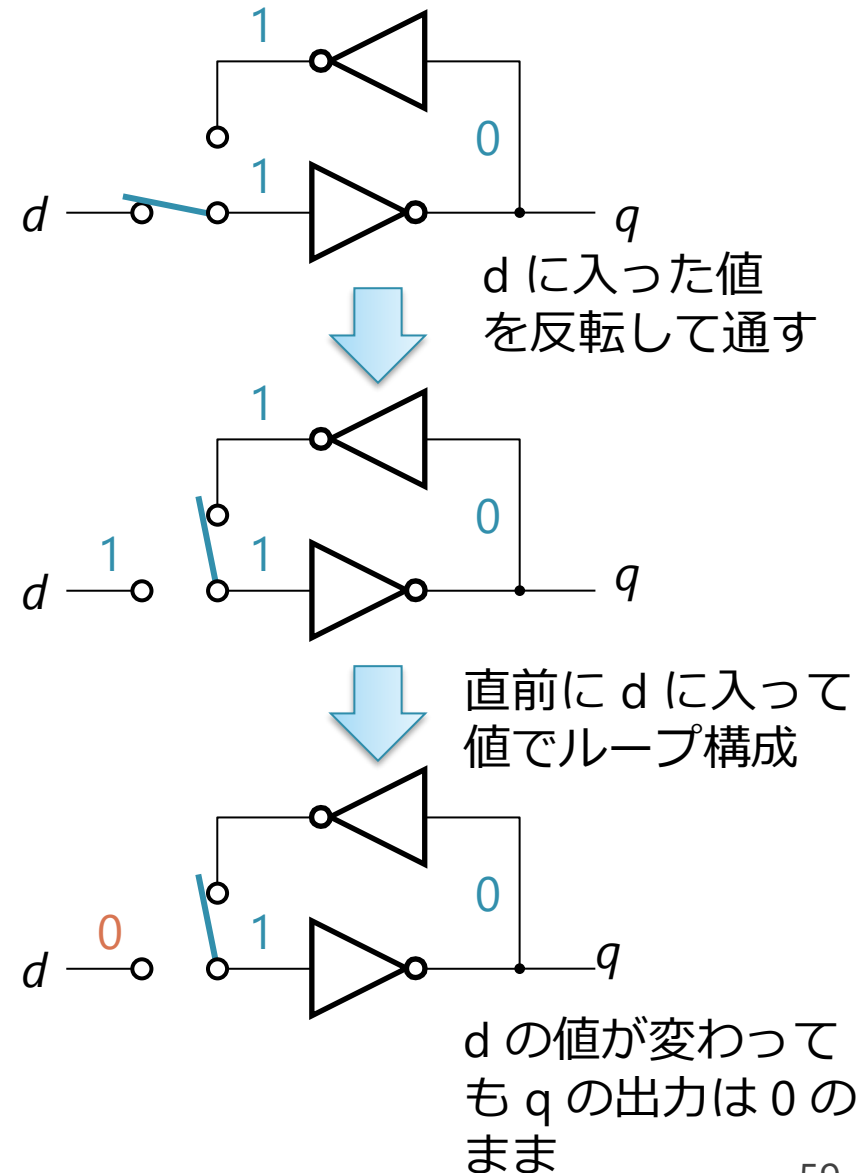
■ 記憶の実現方法：

- 2つの NOT ゲート（インバータ）をループさせた回路により実現
- 以下の2通りの安定状態がある
 - これのどっちになっているによって、1 bit の情報を記憶



D ラッチの回路

- 構造：マルチプレクサが入ったインバータのループ
 - ◇ ここではマルチプレクサを切り替えスイッチとして説明
 - ◇ クロックの立ち上がりのたびに、スイッチが切り替わる
 - ◇ d の値をループに取り入れ、取り入れた値が q から出力される



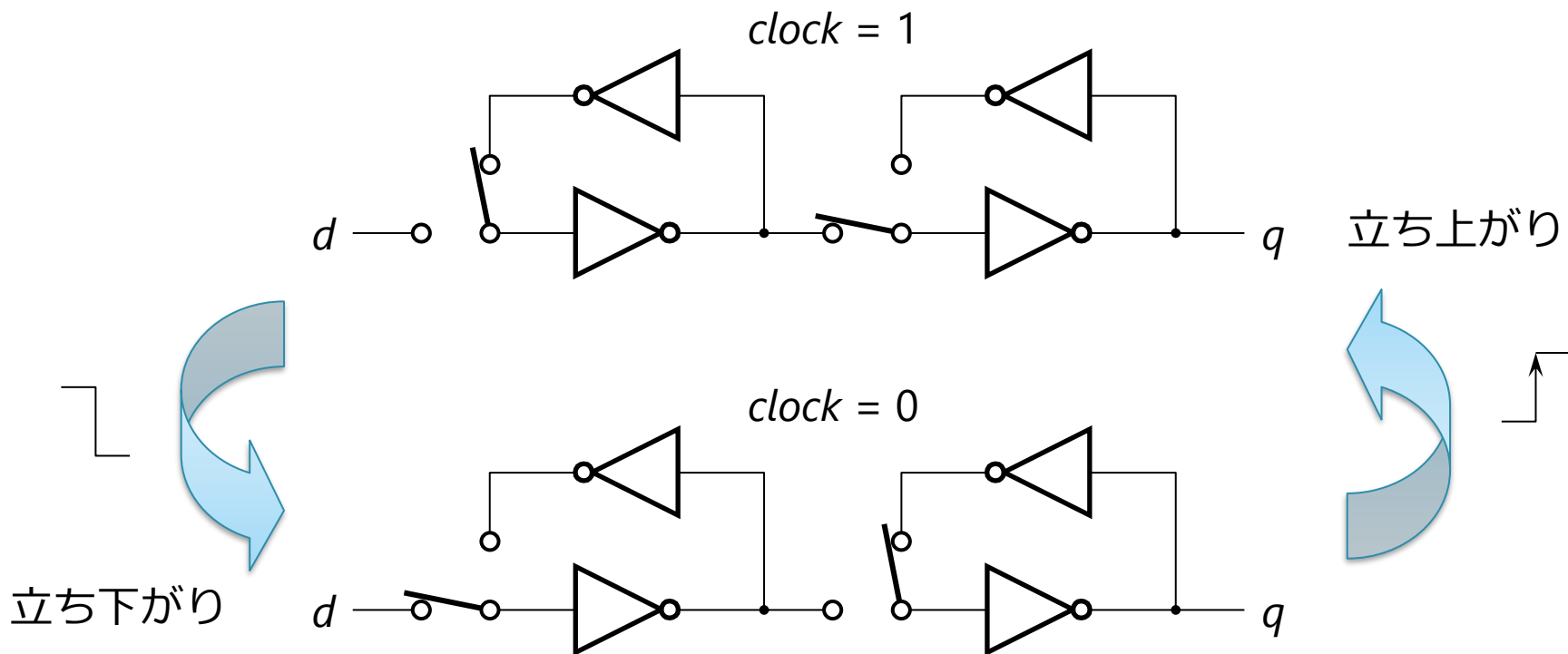
D-FF の回路

■ 構造 : D ラッチを2つ繋げたもの

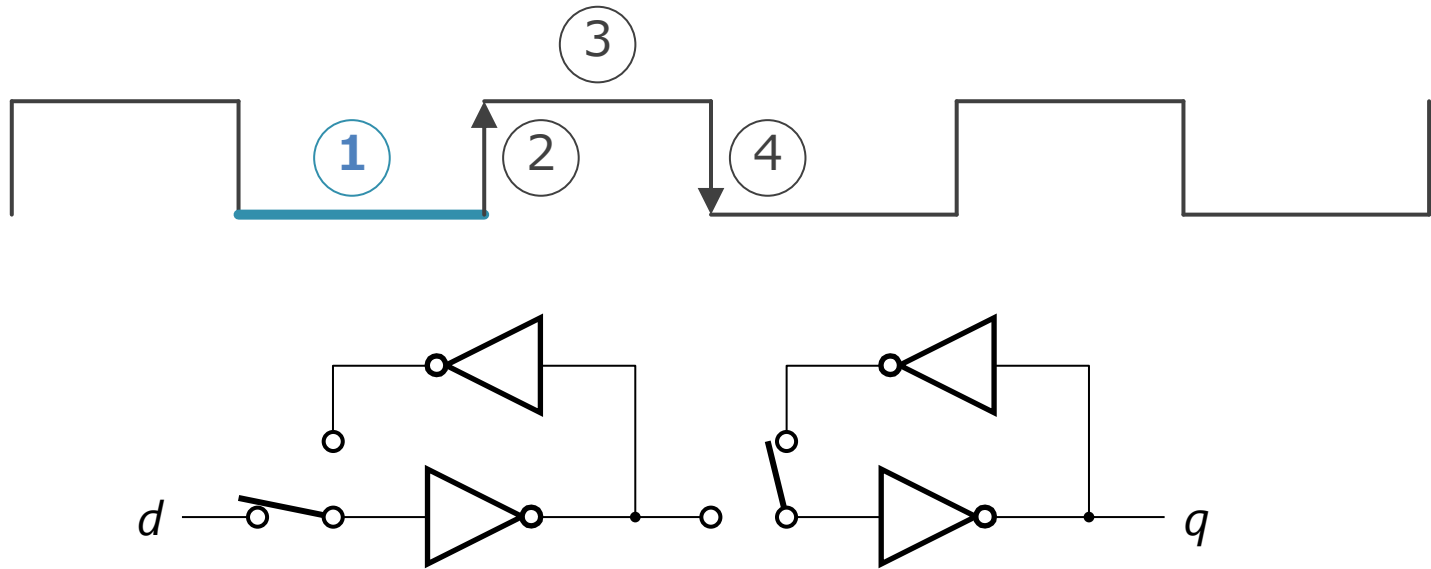
- ◇ D ラッチ 1 つだと, 半周期は d に入った値が反転して素通しするので使いにくい
- ◇ 2 つ直列に繋げて素通しの期間をなくす

■ クロックの立ち上がりのたびに, d の値がサンプリングされる

- ◇ その値が次のサイクルの間 q から出力される



D-FF の動作 ① クロック信号が Low にあるとき



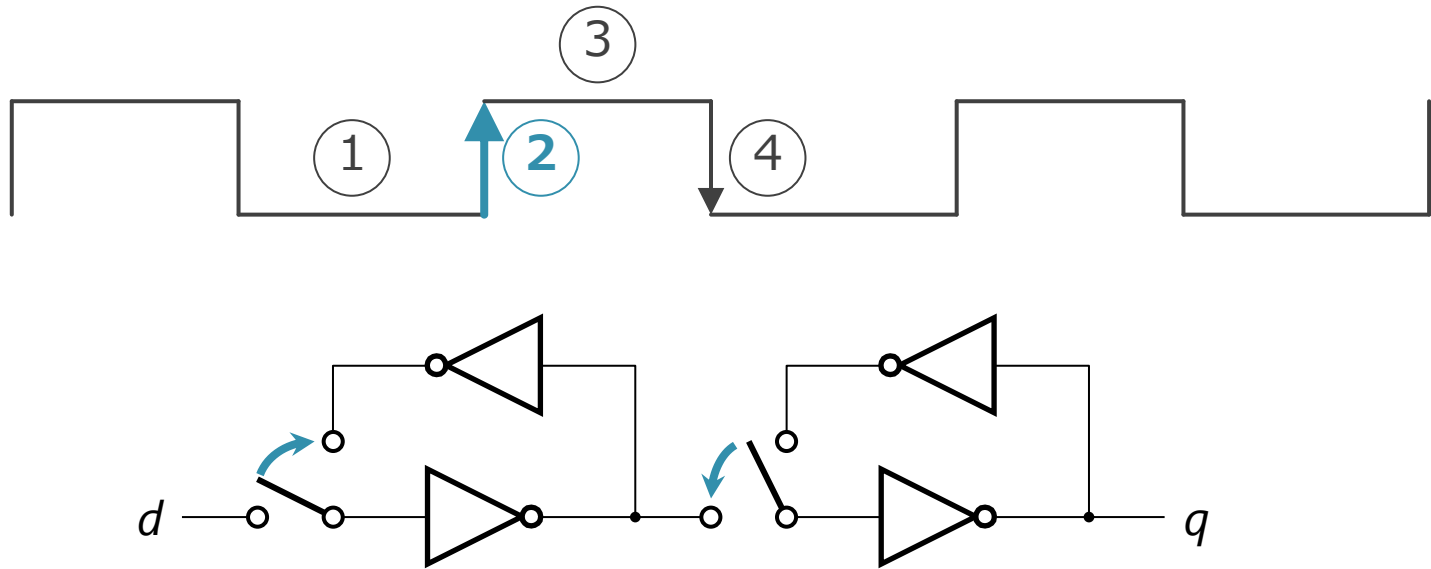
■ 左側のループ :

- ◇ d の入力の変化に応じて, インバータの状態が随時切り替わる
- ◇ 右側のループとは遮断されている

■ 右側のループ :

- ◇ ループのインバータの状態 (=記憶) が q に出力され続ける

D-FF の動作 ② クロック信号の立ち上がり



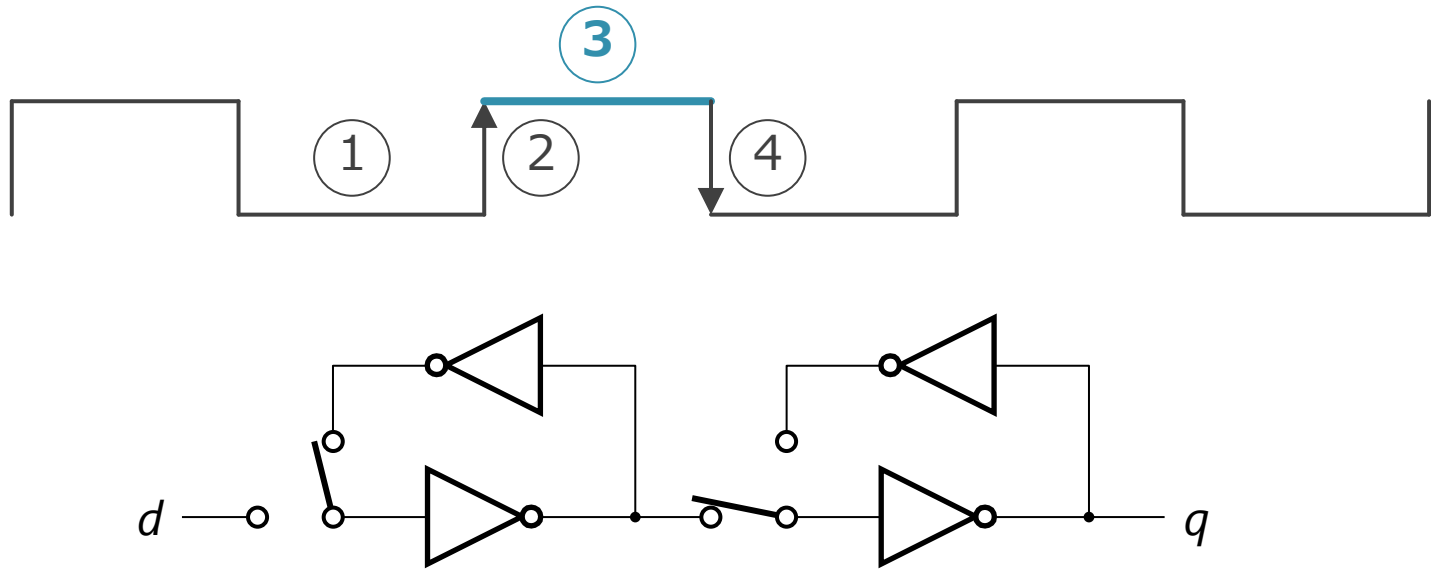
■ 左側のループ :

- ◇ d と遮断され, ループが形成される
- ◇ 直前まで d に入力されていた信号が記憶される

■ 右側のループ :

- ◇ 左側のループと繋がり, ループが解除される

D-FF の動作 ③ クロック信号が High



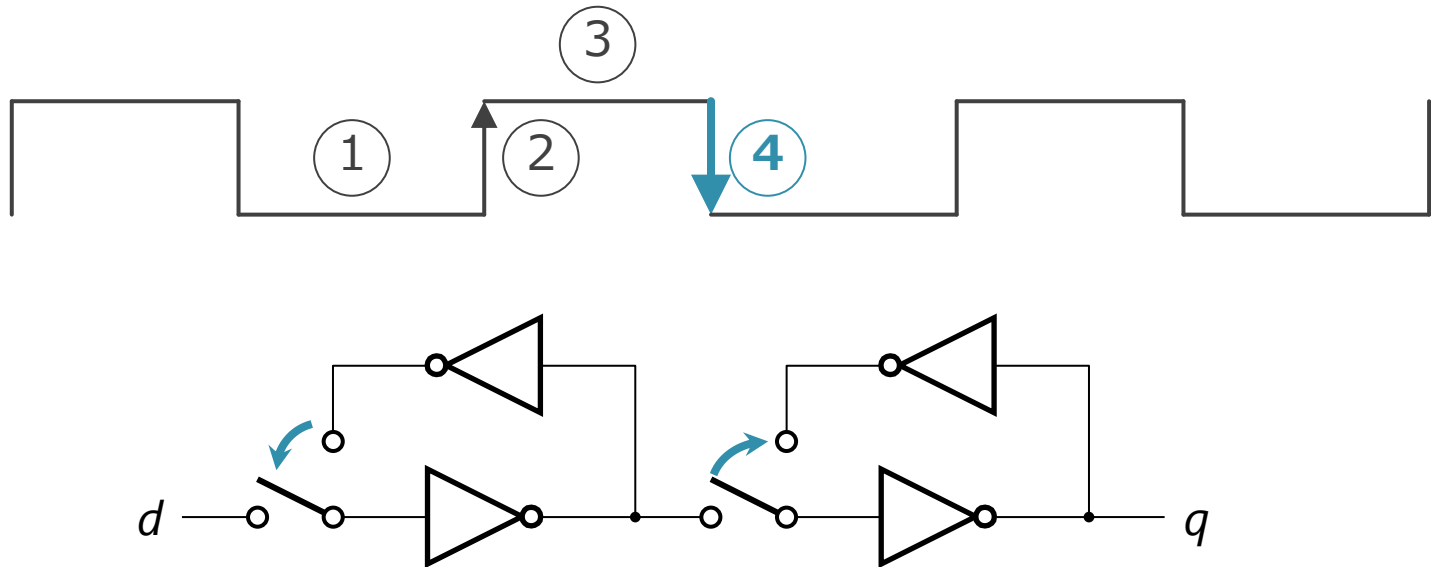
■ 左側のループ :

- ◇ クロックが立ち上がる直前の d の内容を出し続ける

■ 右側のループ :

- ◇ 左側のループの出力を反転して q に出力

D-FF の動作 ④ クロック信号の立ち下がり



■ 左側のループ :

- ◇ ループが解除される

■ 右側のループ :

- ◇ 左側のループと遮断され, ループが形成される
- ◇ それまで左側から入力された内容を出し続ける

D-FF の遅延

- D-FF の遅延：これまでの4フェーズの動作の遅延
 - スイッチが切り替わるまでの遅延
 - スイッチが切り替わった後,
インバータの入力に応じて出力が変化するまでの遅延
- クロック周波数を上げすぎると、これらの限界にぶつかる
 - 1ステージ内の組み合わせ回路の遅延：
インバータ換算で通常10から20段分ぐらい
 - なので、D-FF 自体の遅延は意外とバカにならない

理由 2 : 消費電力と熱

■ クロック周波数を上げる

- → 単位時間あたりの回路全体の充放電の回数が増える
- → 消費電力と、それによって発生する熱がそれだけ増える

1. 電力供給の限界

- CPU のチップの端子から流し込める電流の限界
- オームの法則 : $V=IR$
 - 端子のピンの数で R が決まる

2. 放熱の限界

- 温度の上昇に、放熱が追いつかない

まとめ

1. シングル・サイクル・プロセッサの動作
2. 上記のパイプライン化
3. パイプライン化の性能への影響

課題 5

- 第4回の講義資料を参考に、P型/N型リレーを使って以下を構成せよ

- 2入力 OR ゲート
- 3入力 NOR ゲート

2入力 OR の真理値表

<i>a</i>	<i>b</i>	<i>o</i>
0	0	0
0	1	1
1	0	1
1	1	1

3入力 NOR の真理値表

<i>a</i>	<i>b</i>	<i>c</i>	<i>o</i>
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

ヒント： 以下のNAND ゲートの動作を参考にと良い

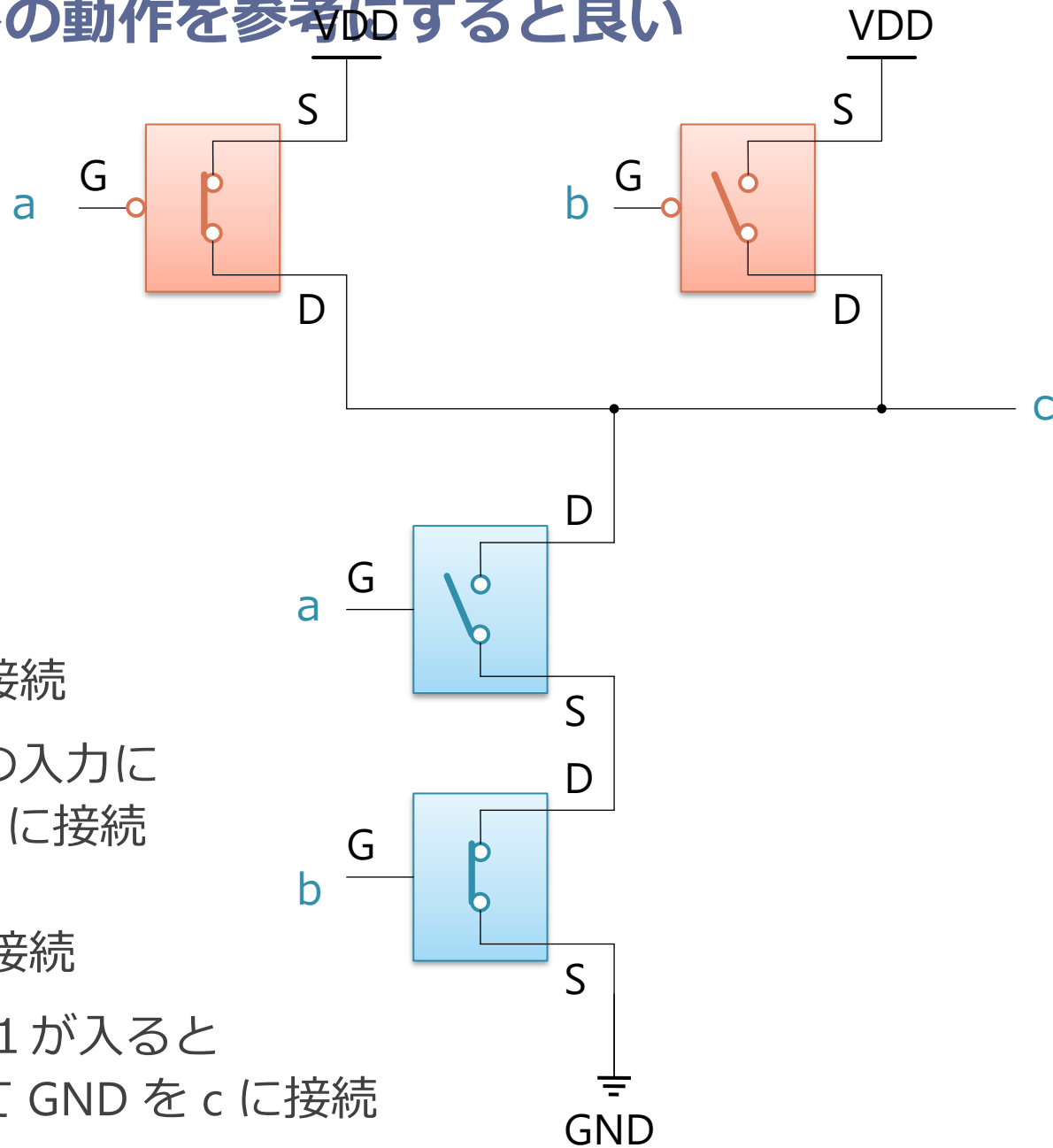
a	b	c
0	0	1
0	1	1
1	0	1
1	1	0

■ 上側の P 型 2 つは並列接続

- a と b どちらか片方の入りに 0 が入ると VDD を c に接続

■ 下側の N 型 2 つは直列接続

- a と b 双方の入りに 1 が入ると 2 つとも ON になって GND を c に接続



提出方法

■ 以下を提出：

1. 課題 5

- 提出は Moodle の「課題 5」のところからお願いします
- 画像（紙に書いてスマホで写真でもよし）で提出してください

2. 感想や質問：

- 「感想や質問」のところに投稿してください
- わからない場所がある場合，具体的に書いてもらえると良いです

■ 提出締め切り

- 来週の講義開始まで

■ 注意：

- 課題の出来は，ある程度努力したあとがあれば良しです
- 必ずしも正解していなくても良いです

次回の講義について

- 6/5（木）は講義日らしい？
- だとしても無理なので，次回は 6/10（火）です

感想や質問

- CMOSゲートの遅延がコンデンサの充放電によるとのことですが、実際の設計ではこの遅延をどう最小化しているのでしょうか？

- CMOSの課題点はどんな研究がなされているのか調べてみたところ、光量子の話が出てきました。光量子は計算速度が速いもののノイズに弱い、大規模な計算回路を作るのが難しい、普通のコンピュータとのインターフェースがなかったり、と様々な課題点があることも知りました。

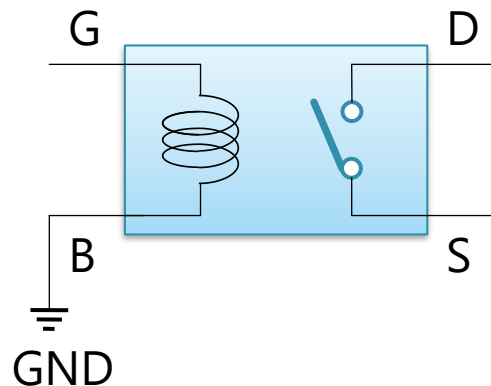
- CPUが負荷に応じて動作周波数と電圧を操作しているのはすごいなと思ったのですが、どのような仕組みで操作しているのでしょうか？

- リレーのN型とP型の部分ですが、SとDが入れ替わるのはなぜですか？
 - s: source, d:drain で、電荷の元と、流し込む先を意味している

- リレーやCMOSは、個々の仕組みはとても単純であるのに、それらを組み合わせることでコンピュータを構成するNOT回路やNAND回路を作ることができるということを知り、とても興味深かったです。

質問や感想など

- 一つ質問なのですが、リレーで入力の値を1や0にするためには、どのようにするのでしょうか。すでにBにはVDDやGNDが接続されていますが、GとBはコイルによってつながっているため、入力のためにGをそれらに接続してもショートは起きないのでしょうか。
- コイルは通常十分抵抗があり、ショートにはならない
- (といっても結構電力は使ってしまう)



- NANDがあれば全てのゲートを作ることができるけれど何回もゲートを経ると遅くなるのでNOT、NAND、AND、ORの4つぐらいは用意した方が良いでしょうと思いました。

質問や感想など

- この科目は期末テストがあると思っているんですけど、具体的な日程はもう決まっていたりしますか？決まっていたら教えて欲しいです。
- すいませんきまってないです

質問や感想など

- リレーとCMOSを比べるとCMOSが断然優秀で全部CMOSでいいじゃないと思ったのですが、リレーはどんなところで使われていますか？
- CMOSは、リレーや真空管に比べて、サイズ・速度・電力・耐久性とさまざまな面で優れていますが、リレーや真空管の方が優れている点はあるのでしょうか。また、現在でもリレーや真空管が使われているものにはどのようなものがあるのか気になりました。

- N型とP型をあえて分ける必要なんてあるのかと安直にも考えていたが、ChatGPTに聞いてみたところ接合することで整流作用やスイッチなどにも使えるそう。必要に応じてそれぞれ使ったり組み合わせたりして有効に活用することがわかった。

質問や感想など

- 私は高校で物理基礎を少し学んだが物理は学んでいないため、今回の講義の電荷や電圧の話はあまりよくわからなかった。
- 今日の講義の内容では、最初はN型リレーとP型リレーは0と1のどちらが入力されればONになるのかということとVDDとGNDはONになった時に0と1のどちらを出力するのかということが混ざってしまってよくわからず複雑だと思っていたが、一つ一つ丁寧に解釈することで仕組みを理解することができ、すっきりした。
- 論理回路は見た目よりもかなり単純な構成になっているのだと驚いた。

質問や感想など

- 私は新設された学部の学生ですが、論理回路についてはANDやNAND、ORなどはコンピュータの講義で軽く教わったので理解できます。真理値表についてもほかの講義で触れました。ただ、専門的な部分まで細かく講義するような授業ではなかったもので、ANDやOR等の意味を理解している程度です。また、高校で物理を習っていないためかコンデンサの部分が理解できませんでした。等価回路の読み方？が分かりませんでした。

- 一部のプログラミング言語ではOSは書けないとのお話でしたが、これはどういうことでしょうか？また、私はC言語を触ったことのないのですが、C言語にしかできないことをもっと教えていただきたいと思いました。

- リレーにN型とP型の二種類があるのはなぜなのでしょう。それぞれのメリットやデメリットなどがあるのでしょうか。
 - 電源 or GND が S に接続されているリレーをそれぞれ用意して P型/N型 とすると説明が楽だからです.
 - また, CMOS と説明をあわせる意図があります

質問や感想など

- 今回の課題について初期値の10はAに代入した一方で、Aから引く1やAとの比較対象になる0はアルファベットに代入せずに第2回課題と同じように数字のままにしました。これらは以下のように代入してもよいのですか？ そうすると行数が増えて複雑になってしまうだけでしょうか。
- li 10 → A
li 1 → B
li 0 → C
LABEL:
sub A, B → A
b A>C, LABEL
- どっちでも大丈夫です

質問や感想など

- 課題についての質問で恐縮ですが、演算子にsubではなく add を用い、インクリメント数を-1にした場合、何か不都合なことはあるのでしょうか。
- こっちで課題について聞いて貰うのは全然 OK です
- add で -1 でも良いです

- 論理回路については、文化情報工学科生は習っている人といない人がいます。選択科目の中に「数理基礎論」など論理回路を扱う授業がありました。

https://tw.ao.ocha.ac.jp/syllabus/index_search.cfm?jugyo=24N0045

- 実際に設計データや部品を見ても論理回路で機器が複雑な処理をできるのが不思議な感じです。どうやって設計するのでしょうか？また初めて 0 から 1 を作った人は凄すぎると思います。

実際の回路生成

```
if code == 100:
    out = rs1 xor rs2
elif code == 110:
    out = rs1 or rs2
else:
    out = rs1 and rs2
```

1. ハードウェア記述言語から，論理関数を生成：

- ナイーブには，入力の全パターンに対して出力を記録した真理値表
 - 実際には表のサイズが爆発して，真理値表ではすぐに破綻
 - 表のサイズ = 2 の入力ビット数乗
- さまざまな効率的な保持方法が提案されており，使用されている
 - Binary Decision Diagram (BDD)

実際の回路生成

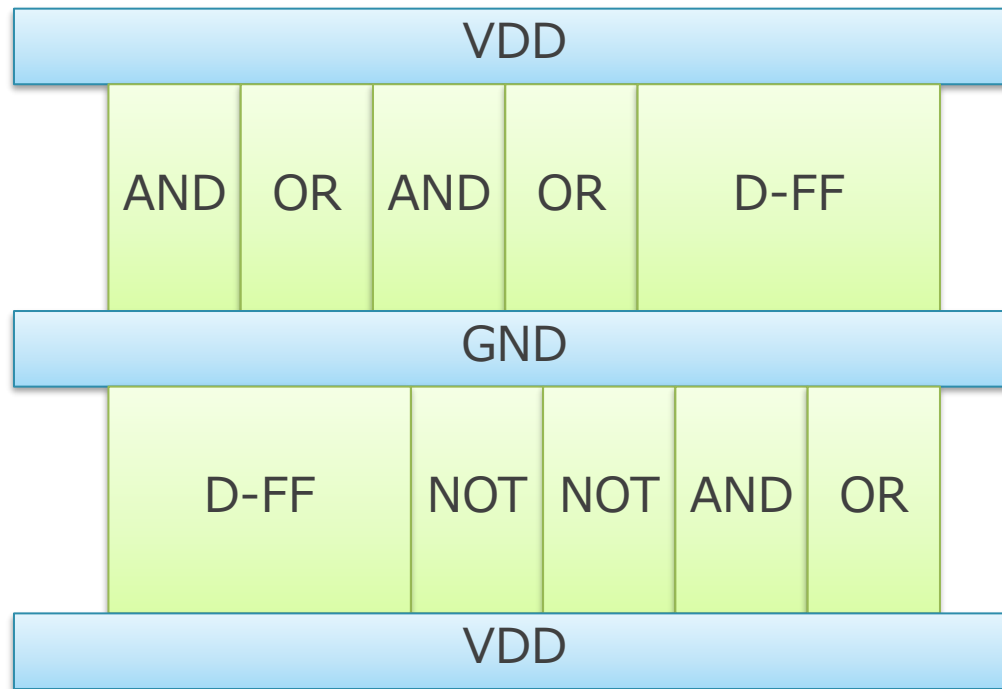
1. 論理合成：

1. ハードウェア記述言語から，論理関数を生成
2. 生成された論理関数を最適化
 - カルノー図：4入力ぐらいが限界
 - クワイン・マクラスキー法：入力数が増えると計算量が爆発
 - さまざまな最適化アルゴリズムが提案・使用されている
3. 回路のプリミティブと接続関係を生成
 - AND, OR, NOT, D-FF を始めとして，よく現れる回路の部品が用意される

2. 配置・配線：

1. 回路のプリミティブを配置・配線

スタンダード・セル

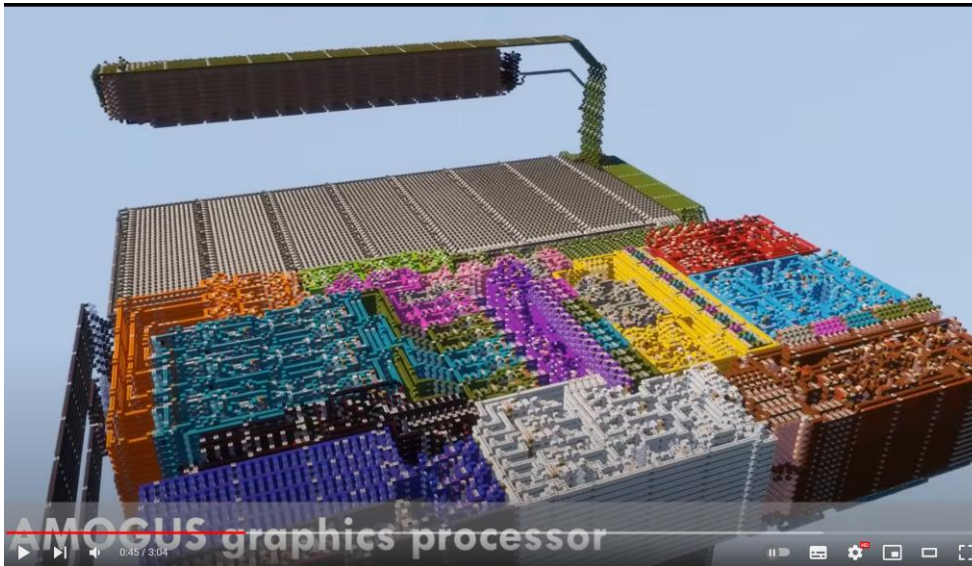


■ スタンダード・セル：

- AND や OR のようなプリミティブとなる回路「セル」を用意
 - 高さを幅を揃えておいて，簡単に並べられるように作ってある
- これらを配置して，配線を接続することにより回路を実現
 - 配置配線という

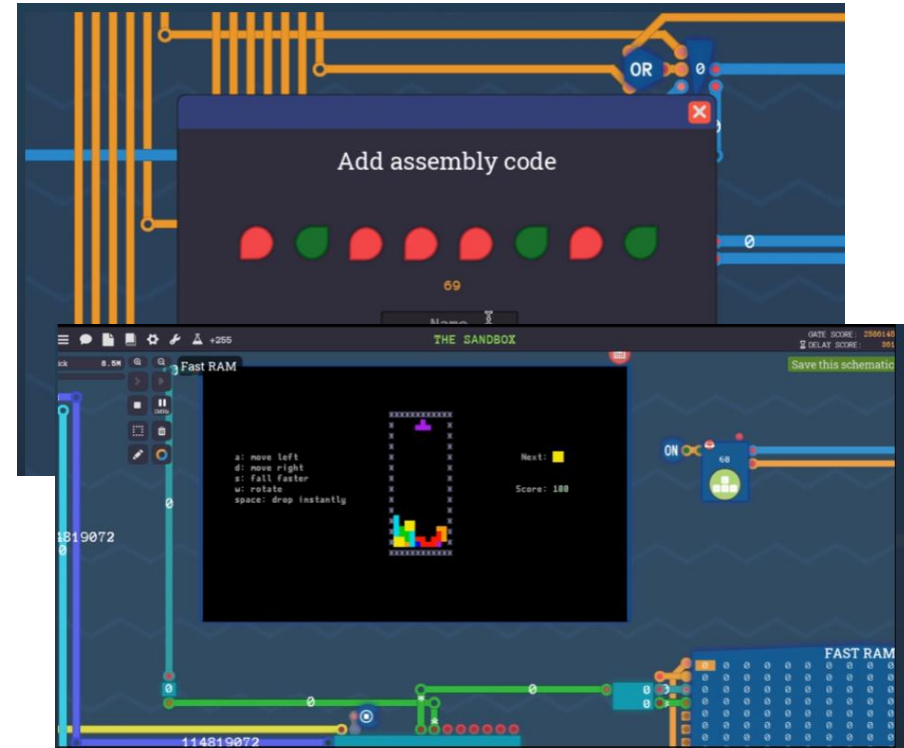
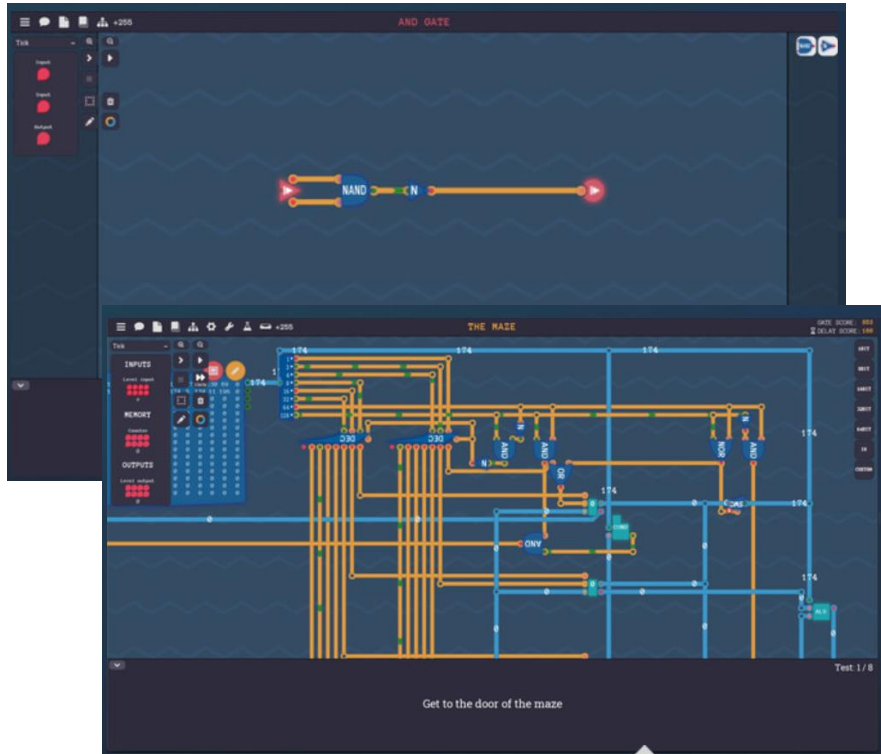
- マインクラフト内でも論理演算ができるということは、まさか頑張ればマインクラフト内で仮想のパソコンを作れるってということですか？

マイクラフトの中でマイクラフト



Turing Complete と言うゲーム

NAND から初めてテトリスとかまでを作る



- https://store.steampowered.com/app/1444480/Turing_Complete/?l=japanese より